

Towards Scalable Fuzzy PSI via Efficient Fuzzy Matching

Meng Hao
Singapore Management University
Singapore
menghao303@gmail.com

Xinpeng Yang
Nanyang Technological University
Singapore
XINPENG004@e.ntu.edu.sg

Hanxiao Chen[✉]
University of Electronic Science and
Technology of China
Chengdu, China
hanxiao.chen@uestc.edu.cn

Tianwei Zhang
Nanyang Technological University
Singapore
tianwei.zhang@ntu.edu.sg

Haiyang Xue
Singapore Management University
Singapore
haiyangxue@smu.edu.sg

Guomin Yang
Singapore Management University
Singapore
gmyang@smu.edu.sg

Hongwei Li[✉]
University of Electronic Science and
Technology of China
Chengdu, China
hongweili@uestc.edu.cn

Robert H. Deng
Singapore Management University
Singapore
robertdeng@smu.edu.sg

Abstract

In fuzzy private set intersection (fuzzy PSI), there are two parties, a sender holding a set of d -dimensional points $Q = \{q_1, \dots, q_m\}$ and a receiver holding a set $W = \{w_1, \dots, w_n\}$ of the same structure. It enables the receiver to learn the point $q \in Q$ for which there exists some $w \in W$ satisfying $\text{dist}(q, w) \leq \delta$ under a given distance metric. Although several fuzzy PSI protocols for $L_{p \in [1, \infty]}$ distance are proposed, there are significant efficiency issues, mainly because they (1) heavily rely on expensive cryptographic primitives, e.g., homomorphic encryption or garble circuits, and/or (2) incur undesirable asymptotic communication and computation complexity.

In this paper, we present scalable fuzzy PSI protocols for general $L_{p \in [1, \infty]}$ distance, supporting both low- and high-dimensional sets. The core technique is two efficient fuzzy matching protocols that securely evaluate $\text{dist}(q, w) \leq \delta$. The first is built from a role-reversed oblivious PRF (OPRF) and realizes $O(d \log \delta)$ overhead, compared to $O((\log \delta)^d)$ in previous works. The second leverages customized oblivious transfer (OT) with $O(d\ell)$ overhead, where ℓ is the bit length of inputs, which is particularly suitable for short inputs. With these new techniques, we further propose a new dual-layer hashing framework for fuzzy PSI over low-dimensional sets, instantiated with our OT-based fuzzy matching and enhanced with a domain reduction optimization. The protocols achieve an overhead linear with $n, m, \log \delta, 2^d$, without the $O((\log \delta)^d)$ or $O(\delta)$ factors present in prior works. For high-dimensional sets, we construct

fuzzy PSI protocols based on our OPRF- and OT-based fuzzy matching, which achieve an asymptotic overhead linear with n, m, d , and $\log \delta$ but rely on the strong globally disjoint assumption.

Extensive evaluations demonstrate that our protocols achieve up to a $145\times$ speedup in running time and a $20\times$ reduction in communication cost compared to van Baarsen and Pu (ASIACRYPT'25), and achieve up to a $25\times$ speedup in running time and up to a $17\times$ reduction in communication cost compared to Piske et al. (CCS'25).

CCS Concepts

• Do Not Use This Code → Generate the Correct Terms for Your Paper.

Keywords

Fuzzy Private Set Intersection, Fuzzy Matching

ACM Reference Format:

Meng Hao, Xinpeng Yang, Hanxiao Chen[✉], Tianwei Zhang, Haiyang Xue, Guomin Yang, Hongwei Li[✉], and Robert H. Deng. 2026. Towards Scalable Fuzzy PSI via Efficient Fuzzy Matching. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 23 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Private set intersection (PSI), including [14, 15, 18, 29, 30, 34–37, 39, 43] and references therein, has been extensively studied over the past decades. It enables a sender S holding a set Q and a receiver R holding a set W to privately compute the intersection $Q \cap W$, while revealing no additional information beyond the output. Traditional PSI protocols focus on *exact* item matching and have been successfully deployed in applications such as private contact discovery [28, 47] and password breach monitoring [20]. However, these protocols are inherently limited in scenarios that require *approximate* matching, such as location-based services and biometric identification.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/06
<https://doi.org/XXXXXXX.XXXXXXX>

To address this gap, a lot of fuzzy PSI protocols [3, 7, 8, 11, 16, 17, 19, 38, 42, 45, 46, 50] have been proposed. In fuzzy PSI, the sender holds a set of d -dimensional points $Q = \{q_1, \dots, q_m\}$ and the receiver holds $W = \{w_1, \dots, w_n\}$ of the same structure. It enables the receiver to learn the point $q \in Q$ for which there exists some $w \in W$ satisfying $\text{dist}(q, w) \leq \delta$ under a given distance metric dist , e.g., Chebyshev distance L_∞ or Minkowski distance L_p . Fuzzy PSI is particularly useful in scenarios where inputs are inherently noisy. For example, in biometric identification, users are matched against biometric samples such as iris scans, fingerprints, or facial patterns. Similarly, in ride-sharing applications, passengers determine whether there are available vehicles within a certain proximity.

A naive approach to fuzzy PSI is to perform pairwise comparisons between elements of the two sets. However, it incurs a quadratic overhead of $O(mn)$ in the set sizes, making it impractical for large-scale sets. To avoid the efficiency issue, existing fuzzy PSI protocols, as illustrated by Gao et al. [16], follow a common paradigm with two phases: coarse mapping and refined filtering. (1) In the coarse mapping phase, initial pairings are established between a sender point q and a receiver point w if they are potentially δ -close, through some *fuzzy mapping* scheme. However, this phase may introduce false positives, that is, two points with a distance greater than δ can still be paired. (2) In the refined filtering phase, a *fuzzy matching* approach is applied to each pair formed during the coarse mapping phase, which determines whether the receiver point w and the sender point q indeed satisfy $\text{dist}(q, w) \leq \delta$. This eliminates the false positives and yields the final results to the receiver.

Recently, a growing body of literature [7, 11, 16, 17, 19, 38, 42, 45, 46, 50] has explored fuzzy PSI under L_p distance for $p \in [1, \infty]$. As detailed in Section 2, we categorize these prior works into low-dimensional and high-dimensional regimes, according to different techniques and designs. In general, these works typically begin by designing *single-point* fuzzy matching protocols, and then introduce fuzzy mapping techniques to compile fuzzy matching into *multi-point* fuzzy PSI. To achieve desirable asymptotic complexities, many recent constructions [11, 16, 45] rely on heavy cryptographic primitives, such as computation-intensive additively homomorphic encryption (AHE) or communication-intensive garbled circuits (GCs). Consequently, these protocols suffer from prohibitive concrete overhead. More recently, van Baarsen and Pu [46] propose the state-of-the-art fuzzy PSI scheme leveraging lightweight oblivious PRFs (OPRFs) to bypass these heavy primitives. Unfortunately, their protocols incur undesirable asymptotic complexities, scaling as $O((\log \delta)^d)$ or $O(\delta)$ for the distance threshold δ and the dimension d . This may introduce severe bottlenecks in real-world applications requiring large distance thresholds. We refer to Table 1 for a comprehensive comparison of existing works.

Motivated by these limitations, we pose the following question:

Can we construct scalable fuzzy PSI protocols with both concrete and asymptotic efficiency guarantees?

1.1 Our Contribution

We answer the above question affirmatively by proposing *scalable* fuzzy PSI for general $L_{p \in [1, \infty]}$ distance over both low- and high-dimensional sets from lightweight oblivious transfer (OT) and oblivious PRF (OPRF). For low-dimensional sets, our protocols achieve

communication and computation complexities that scale linearly with the set sizes m, n and logarithmically with the distance threshold δ , along with an additional exponential factor on the dimension d . This factor also appears in previous works for low-dimensional (small d) settings. For high-dimensional sets, by leveraging stronger assumptions, we achieve an overhead that scales linearly with the dimension d . We summarize our core contributions as follows.

- (1) **Asymptotically efficient fuzzy matching.** We design two asymptotically efficient fuzzy matching protocols. The first is built from our customized role-reversed OPRF and realizes $O(d \log \delta)$ overhead, compared to $O((\log \delta)^d)$ in previous works. The second utilizes customized OT with $O(d\ell)$ overhead, which is particularly suitable for inputs with short bit length ℓ .
- (2) **New framework of fuzzy PSI for low-dimensional sets.** We propose a new fuzzy PSI framework on $L_{p \in [1, \infty]}$ distance from a dual-layer hashing design, incorporating the spatial hashing and Cuckoo hashing. Our framework reduces fuzzy PSI to about m fuzzy matching invocations, which outperforms previous constructions with $2^d n$ invocations. Moreover, we introduce an input space reduction optimization while addressing subtle issues of false positives, which enables the use of our efficient OT-based fuzzy matching.
- (3) **Improved fuzzy PSI for high-dimensional sets.** We propose efficient fuzzy PSI protocols for high-dimensional sets on $L_{p \in [1, \infty]}$ distance based on our OPRF- and OT-based fuzzy matching. Under the strong globally disjoint assumption on both parties' sets, our protocols achieve an asymptotic complexity that is linear in the set sizes m, n and the dimension d , and logarithmic in the distance threshold δ .
- (4) **Extensive experimental evaluations.** We conduct extensive experiments under a wide range of parameter settings. The experimental results demonstrate that our protocols significantly outperform state-of-the-art constructions. In low-dimensional settings, for dimensions d ranging from 2 to 8, our protocols achieve up to 145× faster computation and up to 20× lower communication cost. In high-dimensional settings, for dimensions d ranging from 16 to 64, our protocols require up to 36× less running time and up to 54× less communication.

2 Related Work

In this section, we discuss existing fuzzy PSI protocols for $L_{p \in [1, \infty]}$ distance¹, as they align most closely with our setting. Because the protocol design of fuzzy PSI is largely dictated by the dimensionality of the input sets, we categorize state-of-the-art constructions into low-dimensional and high-dimensional regimes. A comprehensive comparison of these approaches alongside our work is summarized in Table 1.

2.1 Fuzzy PSI on Low-dimensional Sets

Previous fuzzy PSI works for low-dimensional sets instantiate fuzzy mapping using spatial hashing or its variants [19]. The high-level idea is to partition the space into hypercubes of small side length

¹There is also a substantial body of work on fuzzy PSI for Hamming distance [3, 8, 10, 15, 27, 44, 49], which we do not discuss in detail here.

Table 1: Comparisons of existing fuzzy PSI protocols for $L_{p \in [1, \infty]}$ distance, where the sender holds m elements and the receiver holds n elements in a d -dimensional space, and δ is the distance threshold. The protocols of [38, 46] and ours for low-dimensional sets essentially rely on the same assumption, as shown in Appendix A. For clarity, we unify it as the unique ball/center. Note that both [38] and our protocols for low-dimensional sets can be extended to arbitrary receiver sets, but introduce an additional multiplicative factor in complexity.

Setting	Metric	Protocol	Technique	Assumption	Communication	Computation	
						Sender	Receiver
Low Dimension	L_∞	[7]	FSS	$\mathcal{R}$, mini-univ.	$O(dn \log \delta + m2^d (\log \delta)^d)$	$O(m2^d (\log \delta)^d)$	$O((\log \delta)^d n + dm \log \delta)$
		[45]	AHE	$\mathcal{R}$, min $> 2\delta$	$O(\delta dn + 2^d m)$	$O(2^d dm)$	$O(\delta dn + 2^d m)$
			AHE	$\mathcal{R}$, min $> 4\delta$	$O(\delta 2^d dn + m)$	$O(dm)$	$O(\delta 2^d dn + m)$
		[11]	AHE	$\mathcal{R}$, min $> 4\delta$	$O(d \log \delta (n2^d + m))$	$O(2^d dn \log \delta + m(\log \delta)^d)$	$O(dm \log \delta)$
		[42]	GC	$\mathcal{S}$, disj. hash	$O(d \log \delta (n2^s + m2^{d-s}))$	$O(2^{d-s} dm \log \delta)$	$O(2^d dn \log \delta)$
		[38]	OPRF	$\mathcal{R} \wedge \mathcal{S}$, unique ball/center	$O(d(\delta m + 2^d n))$	$O(\delta dm)$	$O(\delta 2^d dn + m)$
		[46]	OPRF	$\mathcal{R} \wedge \mathcal{S}$, unique ball/center	$O(d(m + 2^d n) \log \delta)$	$O(d(m + 2^d n) \log \delta)$	$O(2^d n(d \log \delta + (\log \delta)^{d/2}))$
		Ours	OPRF & OT	$\mathcal{R} \wedge \mathcal{S}$, unique ball/center	$O((m \log \delta + 2^d n)d)$	$O(md \log \delta)$	$O((m \log \delta + 2^d n)d)$
	L_p	[45]	AHE	$\mathcal{R}$, min $> 2\delta(d^{1/p} + 1)$	$O(\delta 2^d dn + \delta^p m)$	$O((d + \delta^p)m)$	$O(\delta 2^d dn + m)$
		[11]	AHE	$\mathcal{R}$, min $> 2\delta(d^{1/p} + 1)$	$O(2^d p(dn \log \delta + m(\log \delta)^d) + dm \log \delta)$	$O(2^d (dpn \log \delta + m(\log \delta)^d))$	$O(m(dp \log \delta + 2^d (\log \delta)^d))$
		[42]	GC	$\mathcal{S}$, disj. hash, $p = 1$	$O(d \log \delta (n2^d + m2^{d-s}))$	$O(dm2^{d-s} \log \delta)$	$O(dn2^s \log \delta)$
			GC	$\mathcal{S}$, disj. hash, $p = 2$	$O(d \log \delta (dn2^d + m2^{d-s}) + \log^3(d\delta)m2^{d-s})$	$O(m2^{d-s}((d \log \delta) + \log^3(d\delta)))$	$O(n2^s(d \log \delta + \log^3(d\delta)))$
		[38]	OPRF	$\mathcal{R} \wedge \mathcal{S}$, unique ball/center	$O(d(\delta m + 2^d n))$	$O(\delta dm)$	$O(d2^d n + m)$
		[46]	OPRF	$\mathcal{R} \wedge \mathcal{S}$, unique ball/center	$O(d(\delta m + 2^d n) + p(m + 2^d n) \log \delta)$	$O(d(\delta + 2^d n) + p(m + 2^d n) \log \delta)$	$O(2^d n(d + p \log \delta))$
		Ours	OPRF & OT	$\mathcal{R} \wedge \mathcal{S}$, unique ball/center	$O((mp \log \delta + 2^d n)d)$	$O(mpd \log \delta)$	$O((mp \log \delta + 2^d n)d)$
High Dimension	L_∞	[45]	AHE	$\mathcal{R}$, disj. proj.	$O((\delta d)^2 n + m)$	$O((\delta d)^2 n + m)$	$O((\delta d)^2 n + m)$
		[16]	AHE	$\mathcal{R} \wedge \mathcal{S}$, disj. proj.	$O(\delta d(m + n))$	$O(\delta dm + n)$	$O(\delta dn + m)$
		[11]	AHE	$\mathcal{R} \wedge \mathcal{S}$, disj. proj.	$O(d(m + n) \log \delta)$	$O(d(m + n) \log \delta)$	$O(d(m + n) \log \delta)$
		[50]	so-OPRF	$\mathcal{R} \wedge \mathcal{S}$, s -separate	$O(\delta^{\frac{d}{s}} m + n)$	$O(\delta^{\frac{d}{s}} m + n)$	$O(\delta^{\frac{d}{s}} n + m)$
		[46]	OPRF	$\mathcal{R} \wedge \mathcal{S}$, global disj.	$O((m\delta + n)d)$	$O((m\delta + n)d)$	$O(nd)$
		Ours	OPRF	$\mathcal{R} \wedge \mathcal{S}$, global disj.	$O((m + n)d \log \delta)$	$O((m + n)d \log \delta)$	$O(nd \log \delta + md)$
	L_p	[16]	AHE	$\mathcal{R} \wedge \mathcal{S}$, disj. proj.	$O(\delta d(m + n) + pm \log \delta)$	$O(\delta dm + pm \log \delta + n)$	$O(\delta dn + pm \log \delta)$
		[11]	AHE	$\mathcal{R} \wedge \mathcal{S}$, disj. proj.	$O(dpn \log \delta + dm \log \delta)$	$O(dpm \log \delta + dn \log \delta)$	$O(dpn \log \delta + dm \log \delta)$
		[50]	so-OPRF	$\mathcal{R} \wedge \mathcal{S}$, s -separate	$O(\delta^{\frac{d}{s}}(m + n) + pm \log \delta)$	$O((\delta^{\frac{d}{s}} + p \log \delta)m + n)$	$O(\delta^{\frac{d}{s}} n + pm \log \delta)$
		[46]	OPRF	$\mathcal{R} \wedge \mathcal{S}$, global disj.	$O((m\delta + n)d + (m + n)p \log \delta)$	$O((m\delta + n)d + (m + n)p \log \delta)$	$O(n(d + p \log \delta))$
		Ours	OPRF & OT	$\mathcal{R} \wedge \mathcal{S}$, global disj.	$O((m + np)d \log \delta)$	$O((m + np)d \log \delta)$	$O(npd \log \delta + md)$

- $\mathcal{R}/\mathcal{S}$ denotes that the set of receiver/sender satisfies the assumption, and $\mathcal{R} \wedge \mathcal{S}$ indicates that both sets satisfy the assumption.
- unique ball/center means in spatial hashing, each grid cell intersects with at most one ball and each grid cell contains at most one point, respectively.
- mini-univ is short for mini-universe assumption, where every ball can be mapped to a distinct vertex of the space.
- min $> l$ means that the minimum distance between any two elements of the set is greater than l .
- disj. hash is short for disjoint hash assumption, which means the spatial hashing scheme maps at most one point to every possible target grid cell.
- disj. proj. is short for disjoint projection assumption, where there exists at least one dimension on which the projection of balls does not overlap.
- global disj. is short for globally disjoint projection assumption, where for every dimension, the projection of balls does not overlap.
- s -separate means for every s dimensions, there exists at least one dimension on which the projection of balls does not overlap.

(also referred to as grid cells), so that the receiver only needs to determine whether any sender's points lie in cells that are δ -close to receiver's points, without enumerating the entire space. Despite this efficiency improvement, spatial hashing-based solutions incur an overhead factor of 2^d , which makes them practical primarily for low-dimensional settings. We summarize these works below according to their underlying technical approaches.

Function secret sharing (FSS). Garimella et al. [19] introduce a general structure-aware PSI framework that can be adapted to fuzzy PSI. The key technique is a customized fuzzy matching protocol built from FSS [5, 6, 21] for testing membership in L_∞ balls. To extend this FSS-based fuzzy matching to fuzzy PSI, they assume that points in the receiver's set are at least 2δ apart in L_∞ distance. Their communication overhead scales as $O((2 \log \delta)^d)$. However, the computation overhead incurs an additional factor of $O(\delta^d)$, since the receiver must traverse all points that are δ -close to each receiver point.

To reduce this computational overhead, Garimella et al. [17] propose an improved fuzzy matching protocol based on incremental FSS. The key idea is to traverse the prefix representation of the receiver's δ -radius ball, which can be represented using only

$(\log \delta)^d$ points. As a result, their fuzzy PSI achieves both computational and communication overheads scaling as $O((2 \log \delta)^d)$. Bui et al. [7] further optimize the concrete efficiency by removing a hidden multiplicative factor proportional to the computational security parameter, while preserving the same asymptotic complexity.

However, the main limitation of FSS-based fuzzy PSI is the overhead factor of $O((2 \log \delta)^d)$, which becomes prohibitive for large distance thresholds. Moreover, these methods only support L_∞ distance, due to the lack of efficient FSS-based constructions for fuzzy matching under general L_p distance.

Additively homomorphic encryption (AHE). To support general $L_{p \in [1, \infty]}$ distance, van Baarsen and Pu [45] propose fuzzy matching protocols leveraging the linear homomorphism of AHE. Their construction incurs $O(d\delta)$ computational and communication overhead. To realize fuzzy PSI, they employ improved spatial hashing, with the assumption that the receiver's points are at least $2\delta(d^{1/p} + 1)$ apart for $L_{p \in [1, \infty]}$ distance. Subsequently, Dang et al. [11] improve the fuzzy matching protocol by integrating prefix trie techniques, reducing communication to $O(\log \delta)$ and computation to $O((\log \delta)^d)$. They adopt the same framework to implement fuzzy PSI for general L_p distances.

However, both of these fuzzy PSI methods suffer from high computational overhead, scaling as $O(d\delta)$ or $O((\log \delta)^d)$, and rely on expensive public-key primitives.

Garbled circuits (GC). To avoid expensive public-key operations, Richardson et al. [42] leverage GC [48], which is built on symmetric primitives, to implement fuzzy matching, and further propose a fuzzy PSI framework based on disjoint hashing, a generalization of spatial hashing. Their framework encodes messages from the GC-based fuzzy matching protocols in an oblivious key-value store [2, 18], which maps a hash identifier for each party's point to the corresponding next GC message. The resulting fuzzy PSI protocols for L_∞ distance achieve $O(m2^{d-s} + n2^s)d \log \delta$ communication, where $s \in [0, d]$ is an integer parameter that can be freely chosen.

However, relying on GCs results in poor concrete communication and computational efficiency for fuzzy PSI, as observed in [38].

Oblivious pseudo-random function (OPRF). Recently, some of the most efficient fuzzy PSI protocols [38, 46] have been constructed using lightweight OPRF techniques. Specifically, van Baarsen and Pu [46] propose OPRF-based fuzzy matching protocols that support general $L_{p \in [1, \infty]}$ distance and further optimize them using prefix trie techniques. To realize fuzzy PSI with these OPRF-based fuzzy matching protocols, they employ spatial hashing and assume that both the sender's and receiver's points are slightly separated. That is the points of both the sender's and receiver's sets are $2\delta d^{1/p}$ or $2\delta(d^{1/p} + 1)$ apart from each other. Despite these optimizations, the resulting fuzzy PSI still incurs overhead scaling as $O((\log \delta)^{d/2})$ for L_∞ distance and $O(\delta)$ for L_p distance.

Concurrently, Piske et al. [38] propose fuzzy matching protocols based on OPRF and oblivious transfer (OT). Their core technique, distance-aware OT, is a variant of OT in which the choice bit depends on whether the sender's and receiver's inputs are close. To achieve fuzzy PSI, they employ spatial hashing, assuming that each grid cell contains at most one sender input. However, unlike other protocols with prefix trie techniques, their protocols incur linear complexity in the distance threshold δ .

2.2 Fuzzy PSI on High-dimensional Sets

To make fuzzy PSI practical for high-dimensional sets, it is necessary to mitigate the exponential overhead regarding d . Several works have addressed this challenge, but they rely on some strong assumptions as discussed below. Similar to low-dimensional sets, we organize these works according to different technical approaches.

Function secret sharing (FSS). To mitigate the 2^d factor in the complexity, Garimella et al. [19] assume that the receiver's δ -radius balls are *globally disjoint*, meaning that the projections of all receiver balls do not overlap in any dimension. Under this assumption, they propose an FSS-based fuzzy PSI protocol that achieves communication complexity scaling to $O(d \log \delta)$, but incurs a computational overhead of $O(\delta^d)$.

Additively homomorphic encryption (AHE). Subsequently, van Baarsen and Pu [45] relax the globally disjoint assumption by introducing the *existing disjoint* assumption. That is, for each receiver's ball, there exists at least one dimension on which its projection does not overlap with that of others. They propose a fuzzy matching protocol based on DDH and extend it to realize fuzzy PSI. However, their protocols' complexity remains quadratic

Functionality $\mathcal{F}_{(\text{ss})\text{FMatch}}$

Parameters: Sender $\mathcal{S}$ and receiver $\mathcal{R}$. Distance metric $\text{dist}(\cdot, \cdot)$ and threshold δ .

Functionality:

- (1) Wait for input $\mathbf{q} \in \mathbb{Z}_{2\ell}^d$ from $\mathcal{S}$ and $\mathbf{w} \in \mathbb{Z}_{2\ell}^d$ from $\mathcal{R}$.
- (2) Compute $z := 1_{\{\text{dist}(\mathbf{q}, \mathbf{w}) \leq \delta\}}$
- (3) FMatch: Output z to $\mathcal{R}$.
- (4) ssFMatch: Sample $z^{\mathcal{S}}, z^{\mathcal{R}} \leftarrow \mathbb{F}_2$ such that $z^{\mathcal{S}} \oplus z^{\mathcal{R}} = z$. Output $z^{\mathcal{S}}$ to $\mathcal{S}$ and $z^{\mathcal{R}}$ to $\mathcal{R}$.

Figure 1: Functionality of (secret-shared) fuzzy matching.

in d . Recently, Gao et al. [16] introduce fuzzy mapping, generalizing spatial hashing-based solutions, and construct fuzzy PSI for high-dimensional sets based on AHE and DDH. Their fuzzy PSI protocol requires that both parties satisfy the *existing disjoint* assumption and realizes communication and computation costs linear in δ, d, m, n . Dang et al. [11] further optimize these protocols using prefix representations, reducing the dependence on δ to logarithmic. Despite these improvements, a key limitation of these works is poor concrete efficiency due to expensive public-key operations.

Oblivious pseudo-random function (OPRF). To address the above efficiency issue, Zhang et al. [50] recently propose an intermediate assumption, which they call the s -separated set assumption. This assumption states that for every s dimensions of each point, there exists at least one dimension on which its projection does not overlap with that of others. When $s = 1$, this degenerates the globally disjoint assumption [19], and when $s = d$, it corresponds to the existing disjoint assumption [45]. They construct fuzzy PSI using shared-output OPRF [1, 13], which rely on two-party computation to evaluate MPC-friendly low-degree PRF, introducing large communication and computation overhead. More recently, relying on the globally disjoint assumption on both parties, van Baarsen and Pu [46] propose the first fuzzy PSI protocol based solely on lightweight OPRF, achieving communication and computation costs that are linear in δ, d, m, n .

3 Preliminaries

3.1 Notations

We use κ and λ to denote the computational and statistical security parameters, respectively. The symbols m and n denote the set sizes of the sender and the receiver, respectively. Bold letters (e.g., $\mathbf{q}$) represent input points with dimension d , and q_k denotes the k -th coordinate of $\mathbf{q}$. For a positive integer a , $[a]$ denotes the set $\{1, \dots, a\}$. The notation $s \leftarrow S$ indicates that s is sampled uniformly at random from S . The indicator function $1_{\{\text{event}\}}$ equals 1 if the event occurs and 0 otherwise.

3.2 Fuzzy Matching and Fuzzy PSI

We introduce the functionality of fuzzy matching and its secret-shared variant in Figure 1 and that of fuzzy PSI in Figure 2 for some distance metric dist on d -dimensional space. Fuzzy matching can be viewed as a simplified single-point version of fuzzy PSI.

Functionality $\mathcal{F}_{\text{FPSI}}$
Parameters: Sender $\mathcal{S}$ and receiver $\mathcal{R}$ with input sizes m, n , respectively. Distance metric $\text{dist}(\cdot, \cdot)$ and threshold δ . Functionality: (1) Wait for input $Q = \{\mathbf{q}_j\}_{j \in [m]} \in \mathbb{Z}_{2^\ell}^{d \times m}$ from $\mathcal{S}$ and $W = \{\mathbf{w}_i\}_{i \in [n]} \in \mathbb{Z}_{2^\ell}^{d \times n}$ from $\mathcal{R}$. (2) Output $Z := \{\mathbf{q}_j \mid \exists i \in [n], \text{dist}(\mathbf{q}_j, \mathbf{w}_i) \leq \delta\}$ to $\mathcal{R}$.

Figure 2: Functionality of fuzzy PSI.

3.3 Prefix Representations of Intervals

We present the prefix trie technique [8, 11, 46] that succinctly and efficiently represents an integer interval.

THEOREM 3.1 ([46]). *Given an integer interval $[q - \delta, q + \delta] \in \mathbb{Z}_{2^\ell}^{2\delta+1}$ and a point $w \in \mathbb{Z}_{2^\ell}$, there are two algorithms with $u, u' = O(\log \delta)$:*

- (1) $\text{PxTrie}(q - \delta, q + \delta) \rightarrow \{\tilde{q}_1, \dots, \tilde{q}_u\}$: The PxTrie algorithm succinctly encodes the interval $[q - \delta, q + \delta]$ into u prefix nodes $\{\tilde{q}_1, \dots, \tilde{q}_u\} \in \mathbb{Z}_{2^\ell}^u$ that cover the entire interval without overlap. Specifically, $u \leq 2 + \log \delta$ when δ is a power of 2.
- (2) $\text{PxPath}(w, \delta) \rightarrow \{\tilde{w}_1, \dots, \tilde{w}_{u'}\}$: The PxPath algorithm expands each query point $w \in \mathbb{Z}_{2^\ell}$ into a path of prefix nodes $\{\tilde{w}_1, \dots, \tilde{w}_{u'}\} \in \mathbb{Z}_{2^\ell}^{u'}$, where $u' := \lfloor \log(2\delta + 1) \rfloor + 1$.

Then, it holds that $\tilde{w}_t \in \{\tilde{q}_1, \dots, \tilde{q}_u\}$ for some unique $t \in [u']$ if and only if $w \in [q - \delta, q + \delta]$.

That means if $w \in [q - \delta, q + \delta]$, there exists a unique $t \in [u']$ such that $\tilde{w}_t \in \{\tilde{q}_1, \dots, \tilde{q}_u\}$; otherwise, $\{\tilde{q}_t\}_{t \in [u]} \cap \{\tilde{w}_t\}_{t \in [u']} = \emptyset$. To simplify the exposition of security proofs, we establish the following three direct corollaries.

COROLLARY 3.2. *Given any two intervals $[q - \delta, q + \delta]$, $[q' - \delta, q' + \delta] \in \mathbb{Z}_{2^\ell}^{2\delta+1}$ such that $|q - q'| > 2\delta$, it holds that $\text{PxTrie}(q - \delta, q + \delta) \cap \text{PxTrie}(q' - \delta, q' + \delta) = \emptyset$.*

PROOF. Suppose for contradiction there exists $z^* \in \text{PxTrie}(q - \delta, q + \delta) \cap \text{PxTrie}(q' - \delta, q' + \delta)$. Let z be any specific value covered by the prefix z^* (e.g., the value obtained by setting all trailing wildcard bits to 0). This means $z \in [q - \delta, q + \delta] \cap [q' - \delta, q' + \delta]$, which implies $|z - q| \leq \delta$ and $|z - q'| \leq \delta$. This yields $|q - q'| = |(q - z) - (q' - z)| \leq |q - z| + |q' - z| \leq 2\delta$. This contradicts the assumption $|q - q'| > 2\delta$. Therefore, the intersection is empty. $\square$

COROLLARY 3.3. *Given any two points $w, w' \in \mathbb{Z}_{2^\ell}$ such that $|w - w'| > 2\delta$, it holds that $\text{PxPath}(w, \delta) \cap \text{PxPath}(w', \delta) = \emptyset$.*

PROOF. Suppose for contradiction there exists $z \in \text{PxPath}(w, \delta) \cap \text{PxPath}(w', \delta)$. By Theorem 3.1, $\text{PxPath}(\cdot)$ outputs u' prefixes with at most $u' - 1$ trailing wildcards, where $u' = \lfloor \log(2\delta + 1) \rfloor + 1$. Since the shared prefix z has at most $u' - 1$ wildcards, this yields $|w - w'| \leq 2^{u'-1} - 1 = 2^{\lfloor \log(2\delta+1) \rfloor} - 1 \leq 2^{\log(2\delta+1)} - 1 = 2\delta$. This contradicts the assumption $|w - w'| > 2\delta$. Therefore, the intersection is empty. $\square$

COROLLARY 3.4. *Given any interval $[q - \delta, q + \delta] \in \mathbb{Z}_{2^\ell}^{2\delta+1}$, for any two points $w, w' \in \mathbb{Z}_{2^\ell}$ such that $|w - w'| > 2\delta$, there exists at most one point $v \in \{w, w'\}$ such that $\text{PxTrie}(q - \delta, q + \delta) \cap \text{PxPath}(v, \delta) \neq \emptyset$.*

Functionality $\mathcal{F}_{\text{OPPRF}}$
Parameters: Sender $\mathcal{S}$ and receiver $\mathcal{R}$ with input sizes m, n , respectively. Functionality: (1) Wait for input $L = \{(k_j, v_j)\}_{j \in [m]} \subseteq \mathbb{Z}_{2^\ell} \times \mathbb{F}$ from $\mathcal{S}$ and $Q = \{q_i\}_{i \in [n]} \subseteq \mathbb{Z}_{2^\ell}$ from $\mathcal{R}$. (2) Sample a random function $F : \mathbb{Z}_{2^\ell} \rightarrow \mathbb{F}$ such that $F(k) = v$ for $(k, v) \in L$. (3) For $i \in [n]$, compute $y_i := F(q_i)$. (4) Output O^F to $\mathcal{S}$ and $\{y_i\}_{i \in [n]}$ to $\mathcal{R}$.

Figure 3: Functionality of oblivious programmable PRF.

PROOF. Suppose for contradiction that both intersections are non-empty. By Theorem 3.1, this implies $w \in [q - \delta, q + \delta]$ and $w' \in [q - \delta, q + \delta]$. This contradicts the assumption that $|w - w'| > 2\delta$. Therefore, at most one point $v \in \{w, w'\}$ can yield a non-empty intersection. $\square$

3.4 Spatial Hashing

To extend single-point fuzzy matching to multi-point fuzzy PSI, prior works [7, 11, 16, 19, 38, 42, 45, 46] partition the entire input space into smaller hypercubes with side length 2δ , also referred to as grid cells. This tiling ensures that each point needs to be matched only against points residing in a bounded number of cells, instead of the whole input space. We formally define this approach, known as *spatial hashing*, below.

We first introduce the definition of a cell and its identifier. A cell is defined as a L_∞ hypercube of side length 2δ . The function $\text{cell}_{2\delta}(\cdot)$ maps a point $\mathbf{q}$ to the identifier of the cell. The identifier of the corresponding cell $C_{\mathbf{q}} := \text{cell}_{2\delta}(\mathbf{q})$ is given by $(\lfloor \frac{q_1}{2\delta} \rfloor \cdot 2\delta, \dots, \lfloor \frac{q_d}{2\delta} \rfloor \cdot 2\delta)$. Spatial hashing consists of two algorithms:

- (1) $\text{cellhash}(\cdot)$: The function maps a point $\mathbf{q} \in \mathbb{Z}_{2^\ell}^d$ into its unique identifier $C_{\mathbf{q}}$ of the cell containing $\mathbf{q}$.
- (2) $\text{ballhash}(\cdot)$: The function maps a point $\mathbf{q} \in \mathbb{Z}_{2^\ell}^d$ into a set of identifiers $\{C_{\mathbf{q},h}\}_{h \in [2^d]}$ of the 2^d cells that intersect the L_∞ ball of radius δ centered at $\mathbf{q}$.

3.5 (Programmable) Oblivious PRF

An oblivious pseudorandom function (OPRF) is a two-party protocol in which the receiver inputs a batch² of queries $\{q_i\}_{i \in [n]}$. The sender obtains a random function F , while the receiver learns only the corresponding evaluations $\{F(q_i)\}_{i \in [n]}$.

Programmable OPRF (OPPRF), introduced by Kolesnikov et al. [30], extends this functionality by allowing the sender to program the function F at designated inputs: for each sender-chosen pair (k_j, v_j) , the functionality enforces $F(k_j) = v_j$, whereas all non-programmed inputs are mapped to random outputs. We present the functionality of OPPRF in Figure 3. The state-of-the-art OPPRF protocols [39, 43] are constructed by combining OPRF with oblivious key-value stores (OKVS) [18]. Our protocols directly use OPPRF as a fundamental building block, instead of OPRF.

²We note that our protocols invoke only multi-query OPRFs. For simplicity, we omit the qualifier "multi-query" in the remainder of the paper.

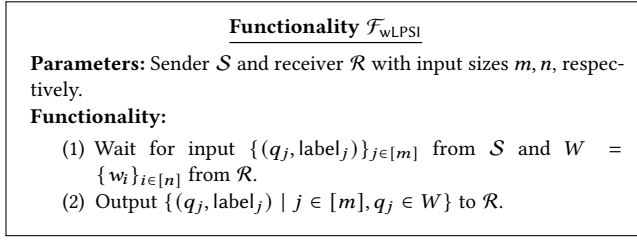


Figure 4: Functionality of weak labeled PSI.

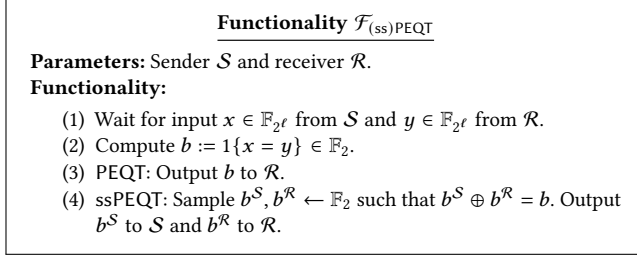


Figure 5: Functionality of (secret-shared) private equality test.

3.6 Weak Labeled PSI

As illustrated in Figure 4, weak labeled PSI [46] is a two-party protocol, where the sender inputs $\{(q_j, \text{label}_j)\}_{j \in [m]}$ and the receiver inputs $W = \{w_i\}_{i \in [n]}$. The receiver learns $\{(q_j, \text{label}_j)\}$ for $q_j \in W$. It differs from the standard notion of labeled PSI in that the functionality outputs the tuple of the intersection item and its corresponding label, instead of just the label. [46] presents an efficient protocol realizing weak labeled PSI from OPRF.

3.7 (Secret-shared) Private Equality Test

The functionality of the private equality test (PEQT) and its secret-shared variant are given in Figure 5.

3.8 Cuckoo Hashing

Cuckoo hashing was introduced by Pagh and Rodler [31]. It maps m items into a table T with $m' = \epsilon m$ bins using k hash functions $H_1, \dots, H_k$. Cuckoo hashing ensures that each bin contains at most one item through the following procedure. For an item x , insert it into any empty bin among $T[H_1(x)], \dots, T[H_k(x)]$. If all such bins are occupied, randomly select one of the k bins, evict the existing item y , and attempt to reinsert y using the same set of hash functions. This process is repeated until all items are successfully placed. Recent works [37] empirically show that with appropriate parameters of ϵ, k , Cuckoo hashing can achieve negligible failure probability.

3.9 Oblivious Permutation

The oblivious permutation functionality receives a secret-shared vector from the two parties. Then, the sender learns a random permutation, and the receiver obtains the permuted vector. The functionality is given in Figure 6. Recent works [9, 22, 32] provide efficient protocols for oblivious permutation and its variants.

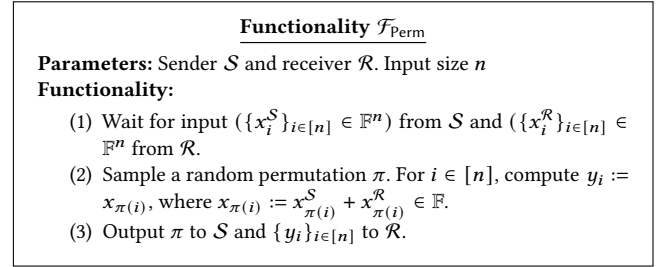


Figure 6: Functionality of oblivious permutation.

4 Technical Overview

In this section, we first introduce efficient fuzzy matching protocols and then present a new framework to compile our single-point fuzzy matching to multi-point fuzzy PSI.

4.1 Asymptotically Efficient Fuzzy Matching

We propose two fuzzy matching protocols for L_∞ distance based on OPRF and OT, respectively. The two protocols target different application scenarios, depending on the bit length of the input points. The extension to L_p distance of OT-based fuzzy matching is provided in Appendix B.2.

Construction from role-reversed oblivious PRF. Our first construction is based on programmable OPRF (OPPRF, see Section 3.5), and achieves $O(d \log \delta)$ computational and communication overhead. This improves upon the state-of-the-art work [46], which also relies on OPPRF but incurs computational overhead $O(d \log \delta + (\log \delta)^{d/2})$.

Our starting point is the OPPRF-based fuzzy matching protocol from [46]. Specifically, for each dimension $k \in [d]$, the sender programs an OPPRF, which maps all points in the interval $[q_k - \delta, q_k + \delta]$ to the same random value r_k . The receiver then obtains the OPPRF evaluation $\hat{r}_k$ on input w_k . Finally, the two parties privately check whether $\sum_{k \in [d]} r_k = \sum_{k \in [d]} \hat{r}_k$. With appropriate parameters, this holds if and only if $w_k \in [q_k - \delta, q_k + \delta]$ for all $k \in [d]$. This elegant protocol incurs an overhead of $O(d\delta)$. The work [46] further optimizes this protocol to achieve logarithmic dependence on δ using the prefix trie technique (see Section 3.3). This enables representing the interval $[q_k - \delta, q_k + \delta]$ with only $u = O(\log \delta)$ elements, hence reducing the number of points programmed by the sender in the OPPRF from $2\delta + 1$ to u . Nevertheless, as a trade-off, the receiver obtains $u' = O(\log \delta)$ OPPRF evaluations $\{\hat{r}_{k,t}\}_{t \in [u']}$ for each dimension k . As a result, the receiver has to compare all possible combinations of $\sum_{k \in [d]} \hat{r}_{k,t_k}$ for $t_k \in [u']$ with the sender's $\sum_{k \in [d]} r_k$. The optimization achieves $O(d \log \delta)$ communication overhead; however, it unfortunately causes a computational overhead of $O((\log \delta)^{d/2})$ even with additional optimizations.

To address this problem, we propose a new fuzzy matching protocol based on two *role-reversed* OPPRF invocations. Specifically, our protocol incorporates prefix trie techniques and executes the first OPPRF like that of [46]. After the receiver obtains OPPRF evaluations $\{\hat{r}_{k,1}, \dots, \hat{r}_{k,u'}\}$ for each dimension k , the parties invoke a second OPPRF with reversed roles to compress these u' evaluations into a single value. More concretely, for each dimension $k \in [d]$, the receiver (as a sender $\mathcal{S}$ in this OPPRF) programs an OPPRF

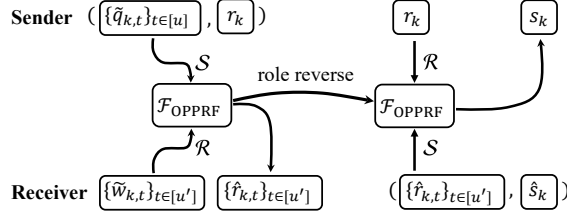


Figure 7: Fuzzy matching from role-reversed OPPRF.

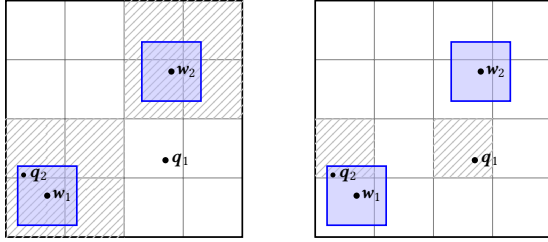


Figure 8: Comparison of fuzzy matching invocations of prior works (left) and ours (right) represented by gray shaded cells.

that maps these u' points $\{\hat{r}_{k,1}, \dots, \hat{r}_{k,u'}\}$ to the same random value $\hat{s}_k$, while the sender (as a receiver $\mathcal{R}$ in this OPPRF) obtains the OPPRF evaluation s_k on input r_k . In Figure 7, we illustrate the two role-reversed invocations of OPPRF for clarity. As shown by our careful analysis in Section 5.1, this construction guarantees that $s_k = \hat{s}_k$ if and only if $w_k \in [q_k - \delta, q_k + \delta]$. Finally, the parties privately check whether $\sum_{k \in [d]} s_k = \sum_{k \in [d]} \hat{s}_k$.

It is worth noting that, although our protocol requires two OPPRF invocations, it maintains $O(d \log \delta)$ communication complexity (with a slightly larger constant factor) and significantly reduces the computational overhead from $O((\log \delta)^{d/2})$ in [46] to $O(d \log \delta)$.

Construction from oblivious transfer. Our second construction is based on OT and incurs $O(d\ell)$ communication and computational overhead, where ℓ denotes the bit length of the input points. When $\ell = O(\log \delta)$, this construction achieves $O(d \log \delta)$ overhead as our OPRF-based approach. Notably, the OT-based construction achieves smaller concrete communication costs, as it eliminates the hidden dependence on the statistical security parameter λ inherently required in OPRF-based constructions [46]. In particular, our OT-based construction leverages recent OT protocols for computing the most significant bit (MSB) and AND [26, 41]. Although technically straightforward and largely overlooked in recent works, this approach achieves highly competitive performance, particularly for small input domains. As demonstrated in our fuzzy PSI protocols for low-dimensional sets, we reduce fuzzy PSI to fuzzy mapping with a small input bit length, namely $\ell = O(\log \delta)$. This significantly reduces communication overhead compared to OPRF-based approaches due to eliminating the factor λ , while incurring only $O(\log \log \delta)$ communication rounds.

4.2 New Framework for Fuzzy PSI

Building upon our single-point fuzzy matching, we now construct multi-point fuzzy PSI protocols. In this section, we focus exclusively

on low-dimensional sets, deferring our high-dimensional constructions to Section 7 and Appendix C. To circumvent the prohibitive $O(mn)$ quadratic overhead inherent to naive pairwise comparisons, we adopt spatial hashing (see Section 3.4) to partition the global d -dimensional input space into grid cells of side length 2δ . For the sender's set, we assume each grid cell contains at most one point. This structural assumption exactly mirrors the setting in [38] and is strictly weaker than the constraints imposed by the majority of prior fuzzy PSI schemes [11, 16, 46, 50]. Furthermore, for clarity of exposition, the following protocols further assume that each grid cell intersects at most one δ -radius ball centered at a receiver point. This constraint aligns with the protocols in [46] and the empirical evaluation framework utilized in [38]. We discuss the extension to arbitrary receiver sets, achieved by leveraging stateful spatial hashing [38, 42], in Section 6.2.

New framework via dual-layer hashing. We present a new fuzzy PSI framework for generalized $L_{p \in [1, \infty]}$ distance, driven by a *dual-layer hashing* strategy that combines spatial hashing with Cuckoo hashing. The core innovation of our approach is an interactive fuzzy mapping phase, which filters the candidate space down to approximately m pairs of points for the final fuzzy matching. This design substantially reduces the overhead of prior OPRF-based constructions [38, 46], which implicitly require $2^d n$ fuzzy matching invocations.

To understand this reduction, we first recall existing fuzzy mapping approaches based on spatial hashing. The sender applies a cellhash function to assign each of its points q to a *unique* cell C_q . Conversely, the receiver applies a ballhash function to associate each of its points w with 2^d distinct cells, $\{C_{w,h}\}_{h \in [2^d]}$, representing all cells intersected by the δ -radius ball centered at w . To find matches, the receiver only needs to compare w against sender points located in these 2^d cells, since any sender point q satisfying $\text{dist}(q, w) \leq \delta$ must reside in one of them. However, when scaled across all n receiver points, this paradigm forces prior protocols to execute approximately $O(2^d n)$ heavy fuzzy matching operations.

Our key observation is that the vast majority of these $2^d n$ receiver search cells are empty on the sender's side, since there are at most m cells in which the sender's points lie. Consequently, the parties only need to perform fuzzy matching on the m specific cells that actually contain the sender's input points. This insight reduces the required number of fuzzy matching invocations from $O(2^d n)$ to just $O(m)$ (as illustrated in Figure 8). To realize this efficiently, we draw inspiration from circuit-based PSI [24, 36, 43] and employ Cuckoo hashing (detailed in Section 3.8) over spatial hashing. Concretely, the sender uses Cuckoo hashing to map its m cell-point tuples (C_q, q) into a table of size $O(m)$, while the receiver applies simple hashing to its $2^d n$ tuples $(C_{w,h}, w)$. Because both parties use identical hash functions, candidate pairs sharing the same cell identifier will collide in the same bin, isolating the valid candidates for fuzzy matching.

To further minimize the number of fuzzy matching invocations per bin, our protocol integrates an OPPRF. Specifically, for each bin i , the receiver programs the OPPRF³ such that, for every tuple

³Our protocol actually performs a single OPPRF invocation over all bins jointly, instead of an OPPRF instance per bin. This avoids the receiver from padding each bin with dummy items to hide the number of items per bin. Similar techniques have been adopted in circuit-based PSI [36, 39, 43].

$(C_{w,h}, \mathbf{w})$ residing in that bin, the function maps the cell identifier $C_{w,h}$ to $\mathbf{w} - \mathbf{r}_i$, utilizing a single random mask $\mathbf{r}_i$ for the entire bin. The sender then queries the OPPRF using its input cell C_q . If a matching cell exists in bin i , the sender successfully learns $\mathbf{w} - \mathbf{r}_i$, while the receiver retains the mask $\mathbf{r}_i$. Together, these two values constitute secret shares of the receiver's point $\mathbf{w}$. As a result, each bin i yields at most one candidate pair: the sender holds its plaintext point $\mathbf{q}$, and the receiver's point $\mathbf{w}$ is secret-shared between them. Otherwise (no matched cells), according to the functionality of OPPRF, secret shares of a random value will be learned. The parties can then process these pairs independently across all bins using a secret-shared variant of our fuzzy matching protocol.

OT-based fuzzy matching with reduced input domain. We instantiate the remaining fuzzy matching using our OT-based construction over a reduced input domain. Building on observations from prior work [7, 17, 19], spatial hashing effectively partitions the large input space into small cells. Specifically, for any sender point $\mathbf{q}$, any potentially δ -close receiver point must reside within the neighboring cells of $\mathbf{q}$. Geometrically, this search space is bounded by a d -dimensional hypercube of side length 6δ (as illustrated in Figure 12). By restricting operations to this reduced domain, as illustrated in Section 4.1, our OT-based fuzzy matching achieves substantially better concrete performance than the OPRF-based alternative.

However, this domain reduction introduces a subtle risk of false positives. Consider a scenario where a cell contains a sender point $\mathbf{q}$, but none of the receiver's δ -radius balls intersect this cell. As illustrated in the above dual-layer fuzzy mapping, the protocol might yield secret shares of a uniformly random point rather than a valid receiver point $\mathbf{w}$. Because the input domain is now artificially small, the probability that this random point accidentally falls within a distance of δ from $\mathbf{q}$ becomes non-negligible. This not only triggers false positives but also risks leaking information about the sender's private input. To resolve this vulnerability, we integrate an additional consistency check. This mechanism explicitly verifies that the cell containing the sender's point $\mathbf{q}$ is identical to the cell intersected by the receiver's point $\mathbf{w}$. This structural guarantee ensures the receiver only learns $\mathbf{q}$ if the fuzzy matching succeeds *and* both points legitimately map to the same cell. We defer the full technical details to Section 6.

In summary, the concrete efficiency gains of our protocol over prior works stem from a synergistic combination of our dual-layer fuzzy PSI framework and our reduced-domain OT-based fuzzy matching. We emphasize that while our fuzzy PSI protocol still relies on OPPRF to process $O(2^d n)$ items, it achieves substantially lower communication and computation overheads compared to prior OPPRF-based constructions [46]. Our construction requires only $O(m)$ OPPRF queries and transmits programmed payloads bounded by $O(2^d n(d \log \delta + \lambda))$ bits due to reduced input domain. In contrast, prior works such as [46] suffer from a multiplicative factor $O(d \log \delta \cdot \lambda)$ in the $O(2^d n)$ invocations of fuzzy matching.

Protocol $\Pi_{\text{FMatch-oprf}}^{L_\infty}$

Parameter: Sender $\mathcal{S}$ and receiver $\mathcal{R}$. Threshold δ . $\ell_1 := \lambda + 2 \log(u' + 1) + \log d$ and $\ell_2 := \lambda$.

Input: $\mathcal{S}$ inputs $\mathbf{q} = (q_1, \dots, q_d) \in \mathbb{Z}_{2^{\ell_1}}^d$ and $\mathcal{R}$ inputs $\mathbf{w} = (w_1, \dots, w_d) \in \mathbb{Z}_{2^{\ell_2}}^d$.

Protocol:

- (1) For $k \in [d]$, $\mathcal{S}$ computes $\{\tilde{q}_{k,t}\}_{t \in [u]} := \text{PxTrie}(q_k - \delta, q_k + \delta)$, and $\mathcal{R}$ computes $\{\tilde{w}_{k,t}\}_{t \in [u']} := \text{PxPath}(w_k, \delta)$.
- (2) For $k \in [d]$, $\mathcal{S}$ samples $r_k \leftarrow \mathbb{F}_{2^{\ell_1}}$. $\mathcal{S}$ and $\mathcal{R}$ invoke functionality $\mathcal{F}_{\text{OPPRF}}$, where $\mathcal{S}$ inputs $\{(k \| \tilde{q}_{k,t}, r_k)\}_{k \in [d], t \in [u]}$ and learns $\mathcal{O}^F$, while $\mathcal{R}$ inputs $\{k \| \tilde{w}_{k,t}\}_{k \in [d], t \in [u']}$ and learns $\{\hat{r}_{k,t} = F(k \| \tilde{w}_{k,t})\}_{k \in [d], t \in [u']}$.
- (3) For $k \in [d]$, $\mathcal{R}$ samples $\hat{s}_k \leftarrow \mathbb{F}_{2^{\ell_2}}$. $\mathcal{S}$ and $\mathcal{R}$ invoke functionality $\mathcal{F}_{\text{OPPRF}}$, where $\mathcal{R}$ as the sender inputs $\{(k \| \hat{r}_{k,t}, \hat{s}_k)\}_{k \in [d], t \in [u']}$ and learns $\mathcal{O}^{F'}$, while $\mathcal{S}$ as the receiver inputs $\{k \| r_k\}_{k \in [d]}$ and learns $\{s_k = F'(k \| r_k)\}_{k \in [d]}$.
- (4) $\mathcal{S}$ computes $s := \sum_{k \in [d]} s_k \in \mathbb{F}_{2^{\ell_2}}$, while $\mathcal{R}$ computes $\hat{s} := \sum_{k \in [d]} \hat{s}_k \in \mathbb{F}_{2^{\ell_2}}$.
- (5) $\mathcal{S}$ and $\mathcal{R}$ invoke functionality $\mathcal{F}_{\text{PEQT}}$, where $\mathcal{S}$ inputs s , while $\mathcal{R}$ inputs $\hat{s}$ and learns $z \in \mathbb{F}_2$.

Figure 9: Fuzzy matching for L_∞ distance from OPPRF

5 Fuzzy Matching

5.1 Fuzzy Matching for L_∞ Distance from Oblivious PRF

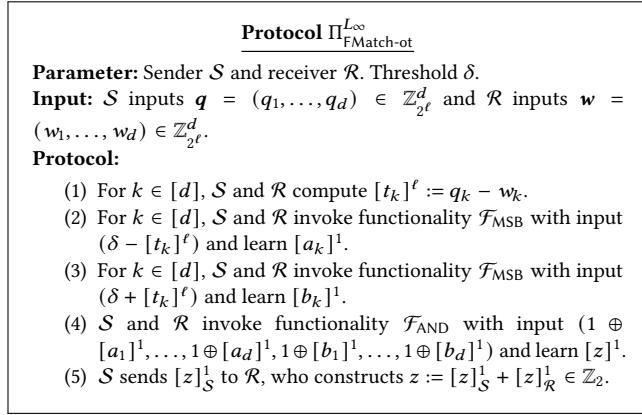
Our first construction is based on OPRF. As the design intuition has already been presented in Section 4.1, we describe the detailed protocol in Figure 9. The key technical contribution is two role-reversed invocations of OPPRF, which enable communication overhead $O(\lambda d \log \delta)$ and computational overhead $O(d \log \delta)$. We note that the use of OPPRF in our protocol can be further optimized by invoking d independent OPPRFs across different dimensions, each involving fewer programmed points and queries. This optimization is formalized as *multi-batch OPPRF* in [46], where batches of points are evaluated under multiple independent random functions.

Extending the above protocol to support L_p distance with the same asymptotic overhead remains challenging and is left as future work, since the use of prefix representations obscures precise distance information. We next state the security of the protocol and defer the detailed correctness and security analysis to Appendix E.1. The key point is to ensure that $\{\hat{r}_{k,t}\}_{t \in [u']}$ are different since they are the programmed points in the second OPPRF invocation, and that $r_k \notin \{\hat{r}_{k,t}\}_{t \in [u']}$ if $w_k \notin [q_k - \delta, q_k + \delta]$ to avoid false positives.

THEOREM 5.1. *The protocol $\Pi_{\text{FMatch-oprf}}^{L_\infty}$ in Figure 9 realizes the functionality $\mathcal{F}_{\text{FMatch}}$ for L_∞ distance in Figure 1 against semi-honest adversaries in the $(\mathcal{F}_{\text{OPPRF}}, \mathcal{F}_{\text{PEQT}})$ -hybrid model.*

5.2 Fuzzy Matching for L_∞ Distance from Oblivious Transfer

Our second fuzzy matching construction is built on OT. To realize fuzzy matching, the goal is to securely evaluate the function $z = 1\{\text{dist}_\infty(\mathbf{q}, \mathbf{w}) \leq \delta\}$, where $\text{dist}_\infty(\mathbf{q}, \mathbf{w}) = \max_{k \in [d]} |q_k - w_k|$. Rather than naively computing this function via absolute-value

Figure 10: Fuzzy matching for L_∞ distance from OT

(ABS) and maximum (MAX) operations followed by a comparison with the threshold δ , we optimize the protocol by directly checking whether $|q_k - w_k| \leq \delta$ for every dimension $k \in [d]$, and then aggregating the resulting comparison bits using AND operations. This approach reduces the overhead: the naive method requires $2d$ MSB operations and $2d$ Multiplexer operations to realize the ABS and MAX functionalities, whereas our optimized protocol only incurs $2d$ MSB operations together with lightweight AND operations.

Our protocol for L_∞ distance is presented in Figure 10. For simplicity, we use $[x]^\ell$ to denote the parties' secret shares $[x]_S^\ell$ and $[x]_R^\ell$ of $x \in \mathbb{Z}_{2^\ell}$. Specifically, our protocol relies on efficient and customized OT-based protocols for computing MSB and AND operations [26, 40, 41]. We describe the functionalities required for OT-based fuzzy matching and summarize the corresponding communication overhead in Appendix B.1. Overall, our protocol incurs $O(d\ell)$ computational and communication overhead.

We extend OT-based fuzzy matching to arbitrary L_p distance in Appendix B.2. Besides, OT-based fuzzy matching protocols are readily extended to realize the secret-shared fuzzy matching functionality in Figure 1 by omitting the last reconstruction step. We next state the security of the protocol and defer the detailed correctness and security analysis to Appendix E.2.

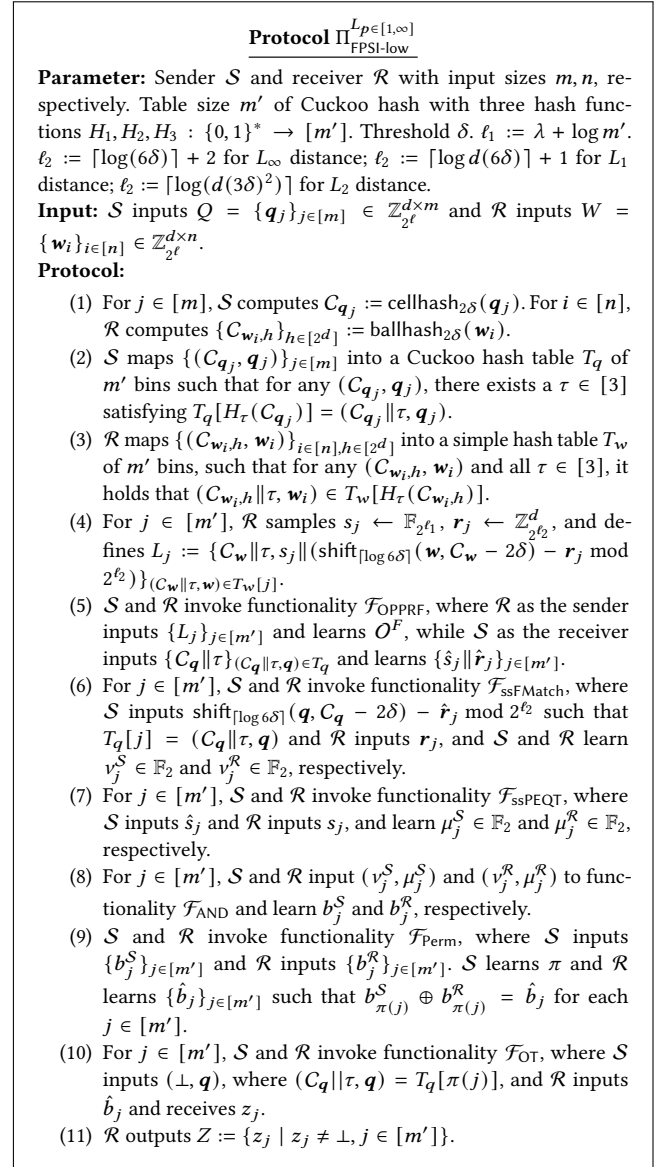
THEOREM 5.2. *The protocol $\Pi_{\text{FMatch-ot}}^{L_\infty}$ in Figure 10 realizes the functionality $\mathcal{F}_{\text{FMatch}}$ for L_∞ distance in Figure 1 against semi-honest adversaries in the $(\mathcal{F}_{\text{MSB}}, \mathcal{F}_{\text{AND}})$ -hybrid model.*

6 Fuzzy PSI over Low-dimensional Sets

6.1 Constructions

We present our fuzzy PSI protocol for $L_{p \in [1, \infty]}$ distance over low-dimensional sets in Figure 11. The protocol relies on assumptions on both the sender's and receiver's sets, which are consistent with those in [46] and the evaluation setup of [11]. That is, for the sender's set, each grid cell contains at most one sender's point; for the receiver's set, each grid cell intersects with at most one δ -radius ball centered at a receiver point. We discuss how to extend to arbitrary receiver sets in Section 6.2.

Since the protocol intuition has already been presented in Section 4, we focus here on providing additional details of the domain-reduction optimization used when invoking fuzzy matching.

Figure 11: Protocol of fuzzy PSI for $L_{p \in [1, \infty]}$ distance over low dimensional sets

Reducing input domain of fuzzy matching. For a sender point $\mathbf{q}$, any receiver point that is potentially δ -close to $\mathbf{q}$ must lie in one of the neighboring grid cells of $\mathbf{q}$. Formally, the neighboring cells of $\mathbf{q}$ are defined as $\{(\mathbf{C}_{\mathbf{q}, 1} + t_1, \dots, \mathbf{C}_{\mathbf{q}, d} + t_d)\}_{t_i \in \{-1, 0, 1\}}$, which collectively form a d -dimensional hypercube with side length 6δ . This significantly reduces the effective input domain for fuzzy matching. To execute fuzzy matching over this reduced domain, we define a *shift-and-truncate* operation as follows:

$$\text{shift}_\ell(\mathbf{q}, \mathbf{o}) := (\mathbf{q} - \mathbf{o}) \bmod 2^\ell,$$

where $\mathbf{o}$ denotes the shift origin, and ℓ is the required bit length to represent points in the reduced domain. In our protocol, for each sender point $\mathbf{q}$, the shift origin $\mathbf{o}$ is chosen as the lower-left corner

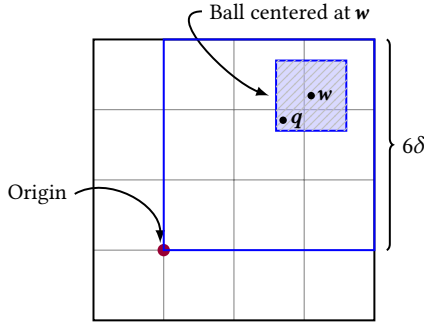


Figure 12: Illustration of reduced input domain after shifting

of the neighboring cells of q , which can be formally expressed as $C_q - 2\delta$. Figure 12 provides an illustrative example. We note that, for a receiver point w , the shift origin is determined by the cell intersected by the δ -radius ball centered at w , rather than by the cell containing w itself.

Addressing false positives caused by small input domain.

Although the above domain-reduction technique significantly reduces the overhead of fuzzy matching, as discussed in Section 4.2, it introduces false positives. In particular, after executing our OPPRF-based fuzzy mapping, the produced pair of points may consist of a sender point q and a uniformly random point, rather than a valid receiver point w . Because the input domain is small, the probability that these two points are δ -close is non-negligible, which may incorrectly reveal q to the receiver.

To mitigate this issue, the receiver appends an additional random value s to the output of each programmed point in the OPPRF. Consequently, the sender obtains the same value s if and only if both parties' inputs correspond to the same cell identifier. We then perform an additional equality test in conjunction with fuzzy matching, which verifies whether the cell containing q is identical to the cell intersected by w . This ensures that the receiver learns q only when the fuzzy matching succeeds *and* the two points are associated with the same cell.

Besides, the ordering of Cuckoo hashing leaks additional information about the sender's entire set. To prevent the leakage, we adopt an oblivious permutation on the fuzzy matching results. The security of the protocol is shown as follows, and we defer the detailed correctness and security analysis to Appendix E.4.

THEOREM 6.1. *The protocol $\Pi_{\text{FPSI-low}}^{L_{p \in [1, \infty]}}$ in Figure 11 realizes the functionality $\mathcal{F}_{\text{FPSI}}$ for $L_{p \in [1, \infty]}$ distance in Figure 2 against semi-honest adversaries in the $(\mathcal{F}_{\text{OPPRF}}, \mathcal{F}_{\text{ssFMatch}}, \mathcal{F}_{\text{ssPEQT}}, \mathcal{F}_{\text{Perm}}, \mathcal{F}_{\text{AND}}, \mathcal{F}_{\text{OT}})$ -hybrid model, if the sender's (resp. receiver's) set satisfies the unique center (resp. ball) assumption.*

6.2 Extension to Arbitrary Receiver's Set

Similar to [38], we can extend our protocols to support arbitrary receiver set distributions without imposing any structural assumptions. To this end, we define a *density parameter* ρ , which denotes the maximum number of receiver balls that intersect the same cell. We then adapt the spatial hashing scheme so that each receiver point is assigned a unique cell identifier. Specifically, we append a

Protocol $\Pi_{\text{FPSI-high}}^{L_{\infty}}$

Parameter: Sender S and receiver R with input sizes m, n , respectively. Threshold δ . Bit lengths $\ell_1 := \lambda + 2 \log(m + nu') + \log d$, $\ell_2 := \lambda + \log m + \log n$.

Input: S inputs $Q = \{q_j\}_{j \in [m]} \in \mathbb{Z}_2^{d \times m}$ and R inputs $W = \{w_i\}_{i \in [n]} \in \mathbb{Z}_2^{d \times n}$.

Protocol:

- (1) For $j \in [m], k \in [d]$, S computes $\{\tilde{q}_{j,k,t}\}_{t \in [u]} := \text{PxTrie}(q_{j,k} - \delta, q_{j,k} + \delta)$.
- (2) For $i \in [n], k \in [d]$, R computes $\{\tilde{w}_{i,k,t}\}_{t \in [u']} := \text{PxPath}(w_{i,k}, \delta)$.
- (3) S samples $\{r_{j,k}\} \leftarrow \mathbb{F}_{2^{\ell_1}}$ for $j \in [m], k \in [d]$. S and R invoke functionality $\mathcal{F}_{\text{OPPRF}}$, where S inputs $\{(k \| \tilde{q}_{j,k,t}, r_{j,k})\}_{j \in [m], k \in [d], t \in [u]}$ and learns O^F , while R inputs $\{(k \| \tilde{w}_{i,k,t}\}_{i \in [n], k \in [d], t \in [u']}$ and learns $\{\hat{r}_{i,k,t} = F(k \| \tilde{w}_{i,k,t})\}_{i \in [n], k \in [d], t \in [u']}$.
- (4) R samples $\hat{s}_{i,k} \leftarrow \mathbb{F}_{2^{\ell_2}}$ for $i \in [n], k \in [d]$. S and R invoke functionality $\mathcal{F}_{\text{OPPRF}}$, where R as the sender inputs $\{(k \| \hat{r}_{i,k,t}, \hat{s}_{i,k})\}_{i \in [n], k \in [d], t \in [u']}$ and learns $O^{F'}$, while S as the receiver inputs $\{(k \| r_{j,k})\}_{j \in [m], k \in [d]}$ and learns $\{s_{j,k} := F'(k \| r_{j,k})\}_{j \in [m], k \in [d]}$.
- (5) For $j \in [m]$, S computes $s_j := \sum_{k \in [d]} s_{j,k}$. For $i \in [n]$, R computes $\hat{s}_i := \sum_{k \in [d]} \hat{s}_{i,k}$.
- (6) S and R invoke functionality $\mathcal{F}_{\text{wLPSI}}$, where S inputs $\{(s_j, q_j)\}_{j \in [m]}$, while R inputs $\{\hat{s}_i\}_{i \in [n]}$ and learns I .
- (7) R outputs $Z := \{z \mid (b, z) \in I\}$.

Figure 13: Protocol of fuzzy PSI for L_{∞} distance over high dimensional sets

counter to the original cell identifier for each receiver ball. Formally, we redefine $\text{ballhash}(w, \rho) = \{C_{w,h} \| D[C_{w,h}] + \}_{h \in [2^d]}$, where D is a dictionary initialized as $D[C] := 1$ for every cell identifier C . This ensures that even when multiple receiver balls map to the same cell, their identifiers remain unique. Correspondingly, for each sender point q , we enumerate all possible matching cell identifiers as $\text{cellhash}(q, \rho) = \{C_q \| h\}_{h \in [\rho]}$. The remainder of the fuzzy PSI protocol remains identical to that shown in Figure 11. As in [38], our original protocol assumes $\rho = 1$. When $\rho > 1$, the complexity of our protocol increases by an additional multiplicative factor of ρ . Therefore, our construction supports fuzzy PSI for arbitrary receiver sets, requiring only the unique cell assumption on the sender's set.

7 Fuzzy PSI over High-dimensional Sets

In this section, we present efficient fuzzy PSI protocols tailored for high-dimensional sets. In this setting, the protocols introduced in Section 6 suffer from an exponential 2^d blowup in overhead, which is a fundamental bottleneck inherent to almost all spatial hashing-based approaches [7, 17, 19, 38, 42, 45, 46]. To bypass this curse of dimensionality, we leverage the *globally disjoint* assumption, previously adopted in [19, 46]. That is, the projections of the sender's and receiver's balls are globally disjoint in every dimension. While our protocols achieve desirable $O((m+n)d \log \delta)$ computation and communication complexity, we acknowledge that this assumption enforces a strong structural constraint and is not entirely realistic.

We formally present the globally disjoint assumption as follows.

Table 2: Comparisons of fuzzy PSI over low-dimensional sets for L_∞ distance. "*" indicates unsupported or no implementation for the specific parameter setting. "-" indicates out of memory or too long running time. The best result is marked in blue.

Set size $m = n$	Dim. d	Protocol	Threshold $\delta = 16$		Threshold $\delta = 64$		Threshold $\delta = 256$		Threshold $\delta = 1024$	
			Time (s)	Comm. (MB)	Time (s)	Comm. (MB)	Time (s)	Comm. (MB)	Time (s)	Comm. (MB)
2^{12}	2	van Baarsen & Pu [46]	0.14	5.65	0.15	7.34	0.21	9.07	0.23	10.76
		Piske et al. [38]	1.92	2.21	1.92	2.21	1.89	2.21	*	*
		Ours	1.00	4.07	0.94	4.07	0.99	4.07	0.96	4.07
	4	van Baarsen & Pu [46]	0.85	35.35	1.08	46.85	1.34	58.53	1.70	70.11
		Piske et al. [38]	4.57	8.98	4.61	8.98	4.48	8.98	*	*
		Ours	1.22	8.57	1.16	8.57	1.22	8.57	1.14	8.57
	8	van Baarsen & Pu [46]	64.27	1053.25	145.74	1401.62	348.73	1755.75	698.87	2106.82
		Piske et al. [38]	33.18	182.91	32.93	182.91	33.07	182.91	*	*
		Ours	4.83	105.06	4.81	105.06	4.84	105.06	4.78	105.06
2^{16}	2	van Baarsen & Pu [46]	1.68	84.14	2.25	111.23	2.83	138.72	3.43	165.97
		Piske et al. [38]	37.46	34.25	38.73	34.25	37.37	34.25	*	*
		Ours	4.92	32.20	5.06	32.20	4.90	32.20	4.96	32.20
	4	van Baarsen & Pu [46]	13.87	560.04	18.89	744.36	24.68	931.54	29.34	1117.24
		Piske et al. [38]	109.56	142.01	87.58	142.01	85.56	142.01	*	*
		Ours	8.46	95.39	8.38	95.39	8.29	95.39	8.35	95.39
	8	van Baarsen & Pu [46]	1278.99	16879.90	2653.76	22465.60	5830.80	28027.10	11697.40	33637.40
		Piske et al. [38]	579.98	2922.36	582.74	2922.36	574.98	2922.36	*	*
		Ours	79.56	1625.46	79.00	1625.46	78.74	1625.46	79.04	1625.46
2^{18}	2	van Baarsen & Pu [46]	6.82	335.67	9.59	444.02	11.93	554.06	14.80	663.20
		Piske et al. [38]	164	136.48	199.41	136.48	*	*	*	*
		Ours	19.55	121.66	19.43	121.66	20.01	121.66	19.57	121.66
	4	van Baarsen & Pu [46]	58.44	2241.18	80.09	2979.16	108.71	3728.65	130.20	4472.26
		Piske et al. [38]	394.26	567.23	399.87	567.23	418.17	567.23	*	*
		Ours	36.20	372.31	35.41	372.31	36.03	372.31	35.89	372.31
	8	van Baarsen & Pu [46]	—	—	—	—	—	—	—	—
		Piske et al. [38]	2591.22	11687.10	2573.38	11687.10	2606.11	11687.10	*	*
		Ours	379.89	6494.57	385.62	6494.57	388.46	6494.57	384.87	6494.57

Definition 7.1 (Globally Disjoint Assumption [19, 46]). A set of d -dimensional balls of radius δ with centers $\mathbf{q}_1, \dots, \mathbf{q}_m$ is globally disjoint if, for each dimension $k \in [d]$, the projections $[q_{j,k} - \delta, q_{j,k} + \delta]$ are disjoint for all $j \in [m]$.

Construction for L_∞ distance. We compile our OPRF-based fuzzy matching protocol to construct fuzzy PSI for L_∞ distance. We do not adopt our OT-based construction, since without spatial hashing it does not admit a sufficiently reduced input domain, leading to large overhead, especially for inputs with large bit length. The idea is that since both parties' sets are globally disjoint, fuzzy PSI protocols can be constructed by using multiple invocations of disjoint OPRF-based fuzzy matching. We present here the protocol in Figure 13, and defer the detailed description to Appendix C.

The security is shown in the following, and its proof is deferred to Appendix E.5.

THEOREM 7.2. *The protocol $\Pi_{\text{FPSI-high}}^{L_\infty}$ in Figure 13 realizes the functionality $\mathcal{F}_{\text{FPSI}}$ for L_∞ distance in Figure 2 against semi-honest adversaries in the $(\mathcal{F}_{\text{OPPRF}}, \mathcal{F}_{\text{wLPSI}})$ -hybrid model if both parties' sets satisfy the globally disjoint assumption.*

Construction for L_p Distance. We further present the fuzzy PSI protocol on high-dimensional sets for L_p distance based on both OPRF- and OT-based fuzzy matching. Our core insight is that the L_∞ protocol described above inherently generates unique identifiers prior to its final output, specifically, s_j for each sender point $\mathbf{q}_j$ and $\hat{s}_i$ for each receiver point $\mathbf{w}_i$. For clarity, we relabel these intermediate values as $\text{id}_{\mathbf{q}_j}$ and $\text{id}_{\mathbf{w}_i}$. By design, a collision $\text{id}_{\mathbf{q}_j} = \text{id}_{\mathbf{w}_i}$ occurs, if and only if $\text{dist}_\infty(\mathbf{q}_j, \mathbf{w}_i) \leq \delta$. Because the

L_∞ ball strictly bounds the L_p ball, these identifiers serve as an effective coarse filter. The parties only need to perform the rigorous $\text{dist}_p(\mathbf{q}_j, \mathbf{w}_i) \leq \delta$ check on candidate pairs that share an identical identifier. To execute this efficiently, we repurpose the Cuckoo hashing-based design from our low-dimensional protocol. However, instead of using spatial grid cells, we use the above intermediate values from the L_∞ protocol in Figure 13 as identifiers. The formal protocol is presented in Figure 14, and additional descriptions are deferred to Appendix C.

The security analysis is shown in the following, and its proof is deferred to Appendix E.6.

THEOREM 7.3. *The protocol $\Pi_{\text{FPSI-high}}^{L_p}$ in Figure 14 realizes the functionality $\mathcal{F}_{\text{FPSI}}$ for L_p distance in Figure 2 against semi-honest adversaries in the $(\mathcal{F}_{\text{OPPRF}}, \mathcal{F}_{\text{Match}}, \mathcal{F}_{\text{OT}})$ -hybrid model if both parties' sets satisfy the globally disjoint assumption.*

8 Evaluation

We provide the full-fledged implementation of our protocols, and our code is available at <https://github.com/ThomasAndy/Scalable-FPSI> and <https://doi.org/10.5281/zenodo.20607518>. We conduct our evaluation on a single server, equipped with two AMD EPYC 9554 processors and 512 GB of RAM. The sender and receiver are emulated as two separate threads within a single process. Each experiment is repeated ten times, and we report the average values. We set the computational security parameter to $\kappa = 128$ and the statistical security parameter to $\lambda = 40$. Network conditions are simulated

Table 3: Comparisons of fuzzy PSI over high-dimensional sets for L_∞ distance. "*" indicates unsupported or no implementation for the specific parameter setting. The best result is marked in blue.

Set size $m = n$	Dim. d	Protocol	Threshold $\delta = 16$		Threshold $\delta = 64$		Threshold $\delta = 256$		Threshold $\delta = 1024$	
			Time (s)	Comm. (MB)	Time (s)	Comm. (MB)	Time (s)	Comm. (MB)	Time (s)	Comm. (MB)
2^{12}	16	van Baarsen & Pu [46]	0.48	47.24	1.66	177.97	6.61	701.40	25.99	2797.59
		Ours	0.42	27.26	0.47	35.37	0.61	43.56	0.70	51.69
	32	van Baarsen & Pu [46]	0.91	94.04	3.42	355.64	13.35	1403.19	54.17	5597.20
		Ours	0.73	53.56	0.95	69.86	1.12	86.35	1.40	102.67
	64	van Baarsen & Pu [46]	1.82	187.68	7.24	711.17	27.93	2807.27	109.52	11199.90
		Ours	1.43	106.39	1.97	138.85	2.50	171.84	3.00	204.53
2^{16}	16	van Baarsen & Pu [46]	7.68	750.94	28.84	2847.02	116.36	11239.50	*	*
		Ours	6.24	424.14	8.06	554.11	10.08	686.16	12.15	817.09
	32	van Baarsen & Pu [46]	15.94	1501.29	62.01	5695.39	261.13	22430.60	*	*
		Ours	12.85	846.85	16.52	1106.96	20.75	1371.10	24.75	1633.09
	64	van Baarsen & Pu [46]	33.03	3002.60	126.33	11395.90	*	*	*	*
		Ours	27.91	1692.64	33.68	2213.14	42.52	2741.69	50.47	3266.10
2^{18}	16	van Baarsen & Pu [46]	32.08	3005.60	140.46	11398.90	*	*	*	*
		Ours	25.36	1695.64	32.86	2216.14	42.69	2744.69	50.47	3269.10
	32	van Baarsen & Pu [46]	66.62	6009.38	328.44	22744.10	*	*	*	*
		Ours	52.51	3388.23	69.22	4429.27	88.19	5487.53	105.49	6536.69
	64	van Baarsen & Pu [46]	136.65	12021.20	*	*	*	*	*	*
		Ours	110.20	6775.05	150.63	8859.02	186.29	10975.40	245.43	13075.30

Protocol $\Pi_{\text{FPSI-high}}^{L_p}$

Parameter: Sender S and receiver R with input sizes m, n , respectively. Threshold δ . Table size n' of Cuckoo hash with three hash functions $H_1, H_2, H_3 : \{0, 1\}^* \rightarrow [n']$. Bit length $\ell' := \lambda + p \log \delta + \log n'$.
Input: S inputs $Q = \{q_j\}_{j \in [m]} \in \mathbb{Z}_{2^{\ell}}^{d \times m}$ and R inputs $W = \{w_i\}_{i \in [n]} \in \mathbb{Z}_{2^{\ell}}^{d \times n}$.

Protocol:

- (1) S and R execute the steps 1-5 of protocol $\Pi_{\text{FPSI-high}}^{L_\infty}$, and learn $\{\text{id}_{q_j}\}_{j \in [m]}$ and $\{\text{id}_{w_i}\}_{i \in [n]}$, respectively.
- (2) S maps $\{(\text{id}_{q_j}, q_j)\}_{j \in [m]}$ into a simple hash table T_q of n' bins, such that for any (id_{q_j}, q_j) and all $\tau \in [3]$, it holds that $(\text{id}_{q_j} \parallel \tau, q_j) \in T_q[H_\tau(\text{id}_{q_j})]$.
- (3) R maps $\{(\text{id}_{w_i}, w_i)\}_{i \in [n]}$ into a Cuckoo hash table T_w of n' bins such that for any (id_{w_i}, w_i) , there exists $\tau \in [3]$ satisfying $T_w[H_\tau(\text{id}_{w_i})] = (\text{id}_{w_i} \parallel \tau, w_i)$.
- (4) S samples $\{g_i\}_{i \in [n']} \leftarrow \mathbb{Z}_{2^{\ell'}}^{d \times n'}$. For $i \in [n']$, S defines $L_i := \{\text{id}_q \parallel \tau, q - g_i \bmod 2^{\ell'}\}$ for all $(\text{id}_q \parallel \tau, q) \in T_q[i]$.
- (5) S and R invoke functionality $\mathcal{F}_{\text{OPPRF}}$, where S inputs $\{L_i\}_{i \in [n']}$ and learns O^G , while R inputs $\{\text{id}_w \parallel \tau\}_{(\text{id}_w \parallel \tau, w) \in T_w}$ and learns $\{\hat{g}_i := G(\text{id}_w \parallel \tau)\}_{i \in [n']}$.
- (6) For $i \in [n']$, S and R invoke functionality $\mathcal{F}_{\text{Match}}$, where S inputs g_i and R inputs $w - \hat{g}_i \bmod 2^{\ell'}$ such that $T_w[i] = (\text{id}_w \parallel \tau, w)$ and learns $b_i \in \mathbb{F}_2$.
- (7) For $i \in [n']$, S and R invoke functionality $\mathcal{F}_{\text{OT}}$, where S inputs $(\perp, g_i)$, and R inputs b_i and receives z_i .
- (8) R outputs $Z := \{z_i + \hat{g}_i \mid z_i \neq \perp, i \in [n']\}$.

Figure 14: Protocol of fuzzy PSI for L_p distance over high dimensional sets

using the Linux `tc` command, configured with a bandwidth of 10 Gbps and a latency of 0.02 ms, following recent works [7, 38, 45].

Implementation details. We provide the implementation details as follows. For OPPrf, we adopt the implementation from [39],

which includes the classic construction based on OKVS and VOLE-OPRF. For OT, we use silent OT from [33]. For MSB, we build on the implementation of [26], replacing their OT component with [33]. All hash functions are instantiated using BLAKE3. For cuckoo hashing parameters, we use 3 hash functions and no stash.

Comparison baselines. We compare our protocols with the state-of-the-art fuzzy PSI schemes [38, 46], as their set assumptions align with ours, and all these protocols rely on OPPrf (and OT). While [38] theoretically supports arbitrary receiver sets, their experimental evaluation adopts the same assumptions as ours and as in [46]. Since [46] does not provide an implementation, for a fair comparison, we implement their constructions with prefix-trie optimizations using the same components as in our protocols. We also incorporate their optimization for solving Knapsack problem [25], which is required in their protocols for L_∞ distances to reduce computation overhead.

8.1 Performance for Low-dimensional Sets

We evaluate our protocols on low-dimensional sets under L_∞ , L_1 , and L_2 distances, and compare them with the state-of-the-art protocols in [38, 46]. The results are reported in Tables 2, 4, and 5. Since [38] supports only 8-bit inputs, we do not report its performance for $\delta = 1024$.

Results of L_∞ distance. As shown in Table 2, our protocol for L_∞ distance demonstrates a significant advantage over [38, 46], especially for relatively large parameter settings. For example, when $m = n = 2^{16}$ and $d = 8$, our protocol is $16 \sim 147 \times$ faster than [46] depending on δ , as their running time grows with $(\log \delta)^{d/2}$. The reason is that the computational overhead of [46] grows as $(\log \delta)^{d/2}$. Although this step is executed in plaintext, when combined with the other components, the overall complexity becomes $n \cdot 2^d \cdot (\log \delta)^{d/2}$, which constitutes a significant bottleneck as both δ and d increase. Moreover, our construction further reduces communication by up

to 20×, since unlike our OT-based fuzzy matching with reduced input domain, the protocol of [46] incur the communication overhead dependent on the statistical security parameter.

Compared to [38], our protocol supports a larger input domain (e.g., 64-bit values in our implementation) while achieving lower running time and communication overhead. Specifically, when $m = n = 2^{18}$ and $d = 8$, our protocol is 7× faster and incurs 1.8× less communication cost than [38]. When extended to larger input domains to support large δ , the overhead of [38] would increase substantially, further highlighting the advantage of our protocols.

Results for L_p Distances. Tables 4 and 5 in Appendix D present the results for L_1 and L_2 distances. We observe that our protocols across the L_∞ , L_1 , and L_2 distance metrics exhibit comparable overall overheads. This is because the constructions execute identical operations, differing exclusively in the $O(m)$ invocations of fuzzy matching (instantiated for either L_∞ or L_p). Due to our input domain reduction optimization, the concrete cost of these fuzzy matching operations does not dominate the total execution time.

Compared to [46], our protocol achieves up to a 5× computational speedup and a 34× reduction in communication bandwidth. Furthermore, we observe a distinct threshold-dependent performance inversion in [46]: for smaller values of δ , their L_∞ protocol incurs higher overhead than their L_p counterpart, whereas for larger δ , their L_p protocol becomes more expensive. This disparity stems from their asymptotic complexities. As outlined in Table 1, the overhead of their L_∞ protocol scales with a multiplicative factor of $2^d d \log \delta$ (dominating for small δ), while their L_p construction is bottlenecked by the terms $d\delta$ (dominating for large δ) and $2^d d$.

Compared to [38], when $m = n = 2^{18}$, $d = 4$, and $\delta = 256$, our protocol achieves up to a 25× speedup and a 17× reduction in communication cost for L_1 distance. Since the protocol in [38] supports only the setting $d = 2$ and $\delta \in \{10, 30\}$ for the L_2 distance, we do not include it in the Table 5. We evaluate our protocol under the same settings ($d = 2$ and $\delta = \{8, 32\}$), and observe that it reduces the communication and computational cost by up to 4× and 10×, respectively.

In some special cases with small parameters, our protocol is less efficient, as the overhead of the underlying silent OT [4] cannot be effectively amortized. Furthermore, the performance of our protocols remains nearly unchanged as δ increases from 16 to 1024, because in practice we round parameters that depend on $\log \delta$ to multiples of 8 to facilitate efficient implementation.

8.2 Performance for High-dimensional Sets

We evaluate our protocols for high-dimensional sets under three distance metrics and compare them with [46], which adopts the same set assumptions as ours and is the state-of-the-art fuzzy PSI for high-dimensional settings.

Results of L_∞ distance. As shown in Table 3, the overhead of our protocol for L_∞ scales logarithmically with δ , outperforming the protocol in [46] across all parameter settings, whose complexity grows linearly with δ . In particular, our protocol exhibits a substantial advantage for large distance threshold δ . When $m = n = 2^{12}$, $d = 64$, and $\delta = 1024$, our protocol requires 36× less running time and 54× less communication than [46]. We note that the protocol in [46] requires a prohibitively large number of OPRF invocations

under some large parameter settings, and no concrete implementation is available for these cases. We therefore mark these entries as “*”.

Results of L_p distances. For the L_1 and L_2 distances, our protocol incurs additional overhead due to the invocation of OT-based fuzzy matching. Nevertheless, it still significantly outperforms [46] for most of the parameter settings. The results are presented in Tables 6 and 7 in Appendix D. Specifically, our protocol achieves up to a 20× reduction in running time and a 38× reduction in communication cost.

9 Conclusion

We present scalable fuzzy PSI protocols for general $L_{p \in [1, \infty]}$ distance, supporting both low- and high-dimensional sets. Our core techniques include two asymptotically efficient fuzzy matching protocols: one based on a customized role-reversed OPRF and another on OT. For low-dimensional sets, we introduce a dual-layer hashing framework that significantly reduces the number of fuzzy matching invocations, while for high-dimensional sets, we enhance existing fuzzy PSI constructions to achieve linear complexity in set size and dimension, with logarithmic complexity in the distance threshold. Overall, our work advances prior fuzzy PSI protocols both concretely and asymptotically.

Ethical Considerations

This work provides effective solutions for fuzzy private set intersection tasks. The experiments in this paper are all based on public datasets and do not contain any personal or illegal information. We firmly believe that our research was done ethically.

Open Science

We follow the principles of the Open Science Policy. Our artifacts include the source code of our protocols, benchmarking scripts, and detailed operation documentation. We have consolidated these research artifacts into an open-source repository, and our artifacts are ready for review and available at <https://github.com/Th0masAndy/Scalable-FPSI> and <https://doi.org/10.5281/zenodo.20607518>. We will openly share our research artifact in the final version of our paper. We grant the scientific community unrestricted access to review, validate, and expand upon our work.

Generative AI Usage

This paper used the LLM (including ChatGPT and Gemini) solely to assist with grammar checking and language polishing.

References

- [1] Navid Alamati, Guru-Vamsi Policharla, Srinivasan Raghuraman, and Peter Rindal. 2024. Improved alternating-moduli PRFs and post-quantum signatures. In *Annual International Cryptology Conference*. 274–308.
- [2] Alexander Bienstock, Sarvar Patel, Joon Young Seo, and Kevin Yeo. 2023. Near-optimal oblivious key-value stores for efficient PSI, PSU and volume-hiding multi-maps. In *USENIX Security Symposium*. 301–318.
- [3] Erik-Oliver Blass and Guevara Noubir. 2025. Assumption-free fuzzy PSI via predicate encryption. *Cryptology ePrint Archive*, Paper 2025/217. <https://eprint.iacr.org/2025/217>
- [4] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, Peter Rindal, and Peter Scholl. 2019. Efficient two-round OT extension and silent non-interactive secure computation. In *ACM SIGSAC Conference on Computer and Communications Security*. 291–308.

- [5] Elette Boyle, Niv Gilboa, and Yuval Ishai. 2015. Function secret sharing. In *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 337–367.
- [6] Elette Boyle, Niv Gilboa, and Yuval Ishai. 2016. Function secret sharing: Improvements and extensions. In *ACM SIGSAC conference on computer and communications security*. 1292–1303.
- [7] Dung Bui, Gayathri Garimella, Peihan Miao, and Phuoc Van Long Pham. 2025. New framework for structure-aware PSI from distributed function secret sharing. In *International Conference on the Theory and Application of Cryptology and Information Security*. 294–326.
- [8] Anrin Chakraborti, Giulia Fanti, and Michael K Reiter. 2023. Distance-aware private set intersection. In *USENIX Security Symposium*. 319–336.
- [9] Melissa Chase, Esha Ghosh, and Oxana Poburinnaya. 2020. Secret-shared shuffle. In *International conference on the theory and application of cryptology and information security*. 342–372.
- [10] Lukasz Chmielewski and Jaap-Henk Hoepman. 2008. Fuzzy private matching. In *International Conference on Availability, Reliability and Security*. 327–334.
- [11] Chengrui Dang, Xv Zhou, and Bei Liang. 2025. Efficient fuzzy PSI based on prefix representation. In *ACM SIGSAC Conference on Computer and Communications Security*.
- [12] Daniel Demmler, Thomas Schneider, and Michael Zohner. 2015. ABY-A framework for efficient mixed-protocol secure two-party computation. In *Network and Distributed System Security Symposium*.
- [13] Itai Dinur, Steven Goldfeder, Tzipora Halevi, Yuval Ishai, Mahimna Kelkar, Vivek Sharma, and Greg Zaverucha. 2021. MPC-friendly symmetric cryptography from alternating moduli: Candidates, protocols, and applications. In *Annual International Cryptology Conference*. Springer, 517–547.
- [14] Changyu Dong, Liqun Chen, and Zikai Wen. 2013. When private set intersection meets big data: an efficient and scalable protocol. In *ACM SIGSAC conference on Computer and communications security*. 789–800.
- [15] Michael J Freedman, Kobbi Nissim, and Benny Pinkas. 2004. Efficient private matching and set intersection. In *International conference on the theory and applications of cryptographic techniques*. 1–19.
- [16] Ying Gao, Lin Qi, Xiang Liu, Yuanchao Luo, and Longxin Wang. 2024. Efficient fuzzy private set intersection from fuzzy mapping. In *International Conference on the Theory and Application of Cryptology and Information Security*. 36–68.
- [17] Gayathri Garimella, Benjamin Goff, and Peihan Miao. 2024. Computation efficient structure-aware psi from incremental function secret sharing. In *Annual International Cryptology Conference*. 309–345.
- [18] Gayathri Garimella, Benny Pinkas, Mike Rosulek, Ni Trieu, and Avishay Yanai. 2021. Oblivious key-value stores and amplification for private set intersection. In *International Cryptology Conference*. 395–425.
- [19] Gayathri Garimella, Mike Rosulek, and Jaspal Singh. 2022. Structure-aware private set intersection, with applications to fuzzy matching. In *Annual International Cryptology Conference*. 323–352.
- [20] Google. 2019. Better password protections in Chrome - How it works. <https://security.googleblog.com/2019/12/better-password-protections-in-chrome.html> Last accessed 11 November 2025.
- [21] Xiaojie Guo, Kang Yang, Xiao Wang, Wenhao Zhang, Xiang Xie, Jiang Zhang, and Zheli Liu. 2023. Half-tree: Halving the cost of tree expansion in COT and DPF. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. 330–362.
- [22] Feng Han, Xiao Lan, Weiran Liu, Lei Zhang, Hao Ren, Lin Qu, and Yuan Hong. 2025. Concretely efficient correlated oblivious permutation. Cryptology ePrint Archive, Paper 2025/449. <https://eprint.iacr.org/2025/449>
- [23] Meng Hao, Hongwei Li, Hanxiao Chen, Pengzhi Xing, Guowen Xu, and Tianwei Zhang. 2022. Iron: Private inference on transformers. *Advances in neural information processing systems* 35 (2022), 15718–15731.
- [24] Meng Hao, Weiran Liu, Liqiang Peng, Hongwei Li, Cong Zhang, Hanxiao Chen, and Tianwei Zhang. 2024. Unbalanced circuit-PSI from oblivious key-value retrieval. In *USENIX Security Symposium*. 6435–6451.
- [25] Ellis Horowitz and Sartaj Sahni. 1974. Computing partitions with applications to the knapsack problem. *J. ACM* 21, 2 (1974), 277–292.
- [26] Zhicong Huang, Wen-jie Lu, Cheng Hong, and Jiansheng Ding. 2022. Cheetah: Lean and fast secure two-party deep neural network inference. In *USENIX Security Symposium*. 809–826.
- [27] Piotr Indyk and David Woodruff. 2006. Polylogarithmic private approximations and efficient matching. In *Theory of Cryptography Conference*. 245–264.
- [28] Daniel Kales, Christian Reibberger, Thomas Schneider, Matthias Senker, and Christian Weinert. 2019. Mobile private contact discovery at scale. In *USENIX Security Symposium*. 1447–1464.
- [29] Vladimir Kolesnikov, Ranjit Kumaresan, Mike Rosulek, and Ni Trieu. 2016. Efficient batched oblivious PRF with applications to private set intersection. In *ACM SIGSAC Conference on Computer and Communications Security*. 818–829.
- [30] Vladimir Kolesnikov, Naor Matania, Benny Pinkas, Mike Rosulek, and Ni Trieu. 2017. Practical multi-party private set intersection from symmetric-key techniques. In *ACM SIGSAC Conference on Computer and Communications Security*. 1257–1272.
- [31] Rasmus Pagh and Flemming Friche Rodler. 2004. Cuckoo hashing. *Journal of Algorithms* (2004), 122–144.
- [32] Stanislav Peceny, Srinivasan Raghuraman, Peter Rindal, and Harshal Shah. 2025. Efficient permutation correlations and batched random access for two-party computation. In *LACR International Conference on Public-Key Cryptography*. 76–109.
- [33] Lance Roy Peter Rindal. [n. d.]. libOTe: an efficient, portable, and easy to use Oblivious Transfer Library. <https://github.com/osu-crypto/libOTe>.
- [34] Benny Pinkas, Mike Rosulek, Ni Trieu, and Avishay Yanai. 2020. PSI from PaXoS: Fast, malicious private set intersection. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. 739–767.
- [35] Benny Pinkas, Thomas Schneider, Gil Segev, and Michael Zohner. 2015. Phasing: Private set intersection using permutation-based hashing. In *USENIX Security Symposium*. 515–530.
- [36] Benny Pinkas, Thomas Schneider, Oleksandr Tkachenko, and Avishay Yanai. 2019. Efficient circuit-based PSI with linear communication. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 122–153.
- [37] Benny Pinkas, Thomas Schneider, and Michael Zohner. 2018. Scalable private set intersection based on OT extension. *ACM Transactions on Privacy and Security* (2018), 1–35.
- [38] Lucas Piske, Jaspal Singh, Ni Trieu, Vladimir Kolesnikov, and Vassilis Zikas. 2025. Distance-aware OT with application to fuzzy PSI. In *ACM SIGSAC Conference on Computer and Communications Security*.
- [39] Srinivasan Raghuraman and Peter Rindal. 2022. Blazing fast PSI from improved OKVS and subfield VOLE. In *ACM SIGSAC Conference on Computer and Communications Security*. 2505–2517.
- [40] Deevashwer Rathee, Mayank Rathee, Rahul Kranti Kiran Goli, Divya Gupta, Rahul Sharma, Nishanth Chandran, and Aseem Rastogi. 2021. Sirnn: A math library for secure rnn inference. In *IEEE Symposium on Security and Privacy*. 1003–1020.
- [41] Deevashwer Rathee, Mayank Rathee, Nishant Kumar, Nishanth Chandran, Divya Gupta, Aseem Rastogi, and Rahul Sharma. 2020. Cryptflow2: Practical 2-party secure inference. In *ACM SIGSAC Conference on Computer and Communications Security*. 325–342.
- [42] David Richardson, Mike Rosulek, and Jiayu Xu. 2024. Fuzzy PSI via oblivious protocol routing. Cryptology ePrint Archive, Paper 2024/1642. <https://eprint.iacr.org/2024/1642>
- [43] Peter Rindal and Philipp Schoppmann. 2021. VOLE-PSI: fast OPRF and circuit-PSI from vector-OLE. In *Annual international conference on the theory and applications of cryptographic techniques*. 901–930.
- [44] Erkam Uzun, Simon P Chung, Vladimir Kolesnikov, Alexandra Boldyreva, and Wenke Lee. 2021. Fuzzy labeled private set intersection with applications to private real-time biometric search. In *USENIX Security Symposium*. 911–928.
- [45] Aron van Baarsen and Sihang Pu. 2024. Fuzzy private set intersection with large hyperballs. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. 340–369.
- [46] Aron van Baarsen and Sihang Pu. 2025. Fuzzy private set intersection from VOLE. In *International Conference on the Theory and Application of Cryptology and Information Security*.
- [47] Mingli Wu and Tsz Hon Yuen. 2023. Efficient unbalanced private set intersection cardinality and user-friendly privacy-preserving contact tracing. In *USENIX Security Symposium*. 283–300.
- [48] Andrew C Yao. 1982. Protocols for secure computations. In *annual symposium on foundations of computer science*. 160–164.
- [49] Qingsong Ye, Ron Steinfeld, Josef Pieprzyk, and Huaxiong Wang. 2009. Efficient fuzzy matching and intersection on private datasets. In *International Conference on Information Security and Cryptology*. 211–228.
- [50] Cong Zhang, Yu Chen, Yang Cao, Yujie Bai, Shuaishuai Li, Juntong Lin, Anyu Wang, and Xiaoyun Wang. 2025. Fast fuzzy PSI from symmetric-key techniques. Cryptology ePrint Archive, Paper 2025/885. <https://eprint.iacr.org/2025/885>

A Assumption of Unique Ball/Center

Our work uses the unique ball/center as the assumption, which is essentially the same as that used in [38, 46] with different representations. For example, [46] assumes that for any $L_p \in [1, \infty]$ distance, the sender's points are $2\delta(d^{1/p})$ -apart and the receiver's points are $2\delta(d^{1/p} + 1)$ -apart. As illustrated in [45, 46], this implies the unique ball/center. We recall their conclusion as follows.

LEMMA A.1 (UNIQUE CENTER [45, 46]). *Suppose there are multiple L_p balls ($p \in [1, \infty]$) with radius δ lying in a d -dimensional space which is tiled by cells with side length 2δ . If these balls' centers are at*

least $2\delta d^{1/p}$ apart from each other, then for each cell, there is at most one center lying in this cell. Specifically, if $p = \infty$, then this holds for disjoint balls, since $2\delta d^{1/p}$ degrades to 2δ in this case.

LEMMA A.2 (UNIQUE BALL [45, 46]). *Suppose there are multiple L_p balls ($p \in [1, \infty]$) with radius δ lying in a d -dimensional space which is tiled by cells with side length 2δ . If these balls' centers are at least $2\delta(d^{1/p} + 1)$ apart from each other, then there exists at most one ball intersecting with the same cell. Specifically, if $p = \infty$, then this holds for L_∞ balls with 4δ -apart centers.*

B Details of OT-based Fuzzy Matching

B.1 Additional Functionalities

Our OT-based fuzzy matching protocols rely on several customized OT-based primitives for securely computing the most significant bit, multiplication, and absolute value. Below, we summarize the corresponding ideal functionalities and their communication overhead of state-of-the-art protocols.

- (1) **Most significant bit (MSB).** The functionality $\mathcal{F}_{\text{MSB}}$ takes as input $[x]^\ell$ and outputs $[y]^1$, where $y = \text{MSB}(x)$. Existing works [26, 41] provide efficient protocols with communication cost below 11ℓ bits and $\log \ell$ communication rounds.
- (2) **AND.** The functionality $\mathcal{F}_{\text{AND}}$ takes as input $\{[x_i]^1\}_{i \in [d]}$ and outputs $[y]^1$, where $y = x_1 \wedge \dots \wedge x_d$. We generate random AND triples using silent OT [4], and then perform Beaver-style multiplications with $4(d-1)$ bits of communication and $\log d$ rounds.
- (3) **Absolute values (ABS).** The functionality $\mathcal{F}_{\text{ABS}}$ takes as input $[x]^\ell$ and outputs $[y]^\ell$, where $y = |x|$. This functionality can be securely realized using MUX and MSB operations [23, 26, 40, 41]. The protocol requires 13ℓ bits of communication and $\log \ell$ rounds.
- (4) **Multiplication (Mult).** The functionality $\mathcal{F}_{\text{Mult}}$ takes as input $([x]^\ell, [y]^\ell)$ and outputs $[z]^\ell$, where $z = x \cdot y$. We employ correlated-OT-based protocols from [12, 40], which incur communication of $\ell^2/2$ bits with 2 rounds.

B.2 OT-based Fuzzy Matching for L_p Distance

We can extend OT-based fuzzy matching to arbitrary L_p distance. In Figure 15, we propose efficient protocols for widely used L_1 and L_2 distances by using MSB, ABS, and multiplication functionalities as shown in Appendix B.1. Our protocol incurs an overhead of $O(p d \ell)$ in both communication⁴ and computation. The security of the protocol is shown as follows, and we defer the detailed correctness and security analysis to Appendix E.3.

THEOREM B.1. *The protocol $\Pi_{\text{FMatch-OT}}^{L_p}$ in Figure 15 realizes the functionality $\mathcal{F}_{\text{FMatch}}$ for L_p distance in Figure 1 against semi-honest adversaries in the $(\mathcal{F}_{\text{ABS}}, \mathcal{F}_{\text{Mult}}, \mathcal{F}_{\text{MSB}})$ -hybrid model.*

C Details of Fuzzy PSI for High-dimensional Sets

⁴We note that the OT-based multiplication protocol incurs $O(\ell^2)$ communication. However, an alternative solution is to employ efficient RLWE-based additively homomorphic encryption, which incurs $O(\ell)$ communication. Therefore, we maintain the $O(p d \ell)$ communication overhead in our asymptotic analysis.

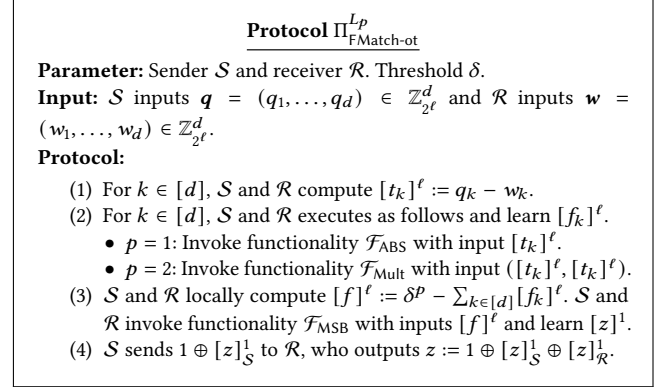


Figure 15: Protocol of fuzzy matching for L_p distance from OT

In this section, we introduce the details of our fuzzy PSI for high-dimensional sets.

Fuzzy PSI for L_∞ distance. We compile our OPRF-based fuzzy matching protocol to construct fuzzy PSI for L_∞ distance. Our protocol follows a similar high-level framework to that of [46]; however, a key distinction is that our construction supports prefix-trie techniques in combination with our asymptotically efficient fuzzy matching. As discussed in [46], prefix-trie techniques are not incorporated in their protocol because doing so incurs a computation cost of $(\log \delta)^{d/2}$, which becomes prohibitively large in high-dimensional settings.

Recall that in our OPRF-based fuzzy matching protocol, for each dimension k , the OPRF programs (a prefix representation of) the interval $[q_k - \delta, q_k + \delta]$ to a single random value r_k . To extend this construction to multi-point fuzzy PSI, the natural approach is to perform OPRF evaluations over all sender and receiver points. However, there are two important requirements. First, the intervals $[q_k - \delta, q_k + \delta]$ and $[q'_k - \delta, q'_k + \delta]$ corresponding to two distinct sender points cannot overlap, since OPRF cannot program the same input to two different values r_k and r'_k . Second, any two receiver queries w_k and w'_k must not fall into the same programmed interval $[q_k - \delta, q_k + \delta]$; otherwise, the receiver would obtain the same value r_k for both queries, which leaks additional information about partial matches on individual dimensions. Fortunately, the globally disjoint assumption satisfies these requirements. Under this assumption, after invoking our role-reversed OPRF, the sender obtains an identifier id_{q_j} for each $q_j \in Q$, and the receiver obtains an identifier id_{w_i} for each $w_i \in W$, such that $\text{id}_{q_j} = \text{id}_{w_i}$ if and only if $\text{dist}_\infty(q_j, w_i) \leq \delta$. Finally, both parties engage in a labeled PSI protocol (see Section 3.6), which enables the receiver to obtain q_j corresponding to the matched identifiers.

Fuzzy PSI for L_p distance. We further extend the above protocol to L_p distance. The above protocol for L_∞ distance before the last weak label PSI can be viewed as a fuzzy mapping operation. This operation assign an unique identifier id_{q_j} for each q_j , similarly id_{w_i} for each w_i , satisfying $\text{id}_{q_j} = \text{id}_{w_i}$ if $\text{dist}_\infty(w_i, q_j) \leq \delta$. To precisely determine the L_p distance, both parties need to execute fuzzy matching on the pairs sharing the same identifier. Since our OPRF-based fuzzy matching can not support L_p distance, we use our

Table 4: Comparisons of fuzzy PSI over low-dimensional sets for L_1 distance. "*" indicates unsupported or no implementation for the specific parameter setting. The best result is marked in blue.

Set size $m = n$	Dim. d	Protocol	Threshold $\delta = 16$		Threshold $\delta = 64$		Threshold $\delta = 256$		Threshold $\delta = 1024$	
			Time (s)	Comm. (MB)	Time (s)	Comm. (MB)	Time (s)	Comm. (MB)	Time (s)	Comm. (MB)
2^{12}	2	van Baarsen & Pu [46]	0.11	7.33	0.27	23.9	0.91	89.48	3.35	351.33
		Piske et al. [38]	1.95	5.9	2.23	13.88	3.59	43.73	*	*
		Ours	0.66	3.17	0.64	3.17	0.79	3.17	0.75	3.17
	4	van Baarsen & Pu [46]	0.27	18.01	0.63	51.04	1.94	182.15	7.02	705.97
		Piske et al. [38]	4.63	16.35	5.36	32.28	8.26	91.96	*	*
		Ours	0.87	6.84	0.89	6.84	0.89	6.84	0.86	6.84
	8	van Baarsen & Pu [46]	4.78	198.62	5.7	264.58	9.17	526.82	20.15	1575.00
		Piske et al. [38]	34.56	197.63	35.54	229.47	40.62	348.8	*	*
		Ours	4.73	101.69	4.64	101.69	4.38	101.69	3.60	134.77
2^{16}	2	van Baarsen & Pu [46]	1.45	111.14	4.06	376.74	14.69	1428.29	58.81	5626.30
		Piske et al. [38]	39.46	93.03	46.25	220.31	79.94	697.43	*	*
		Ours	3.81	27.32	3.94	27.32	3.96	27.32	4.20	27.32
	4	van Baarsen & Pu [46]	4.76	282.25	10.61	811.74	33.35	2913.84	168.91	11312.50
		Piske et al. [38]	91.97	259.52	109.06	514.02	159.56	1468.18	*	*
		Ours	6.72	85.73	6.9	85.73	6.69	85.73	6.61	85.73
	8	van Baarsen & Pu [46]	92.57	3177.7	109.22	4235.31	205.05	8439.44	485.63	25184.90
		Piske et al. [38]	592.08	3157.32	623.35	3666.25	734.61	5574.42	*	*
		Ours	78.13	1606.23	77.66	1606.23	77.7	1606.23	68.39	2136.62
2^{18}	2	van Baarsen & Pu [46]	5.83	443.64	17.82	1507.24	63.13	5717.38	264.72	22468.90
		Piske et al. [38]	171.61	371.44	196.34	880.37	323.33	2788.54	*	*
		Ours	15.81	104.72	16.30	104.72	16.40	104.72	15.98	104.72
	4	van Baarsen & Pu [46]	19.89	1128.75	46.41	3248.83	146.63	11665.30	*	*
		Piske et al. [38]	411.09	1037.06	465.84	2054.81	748.99	5870.97	*	*
		Ours	28.64	338.53	28.71	338.53	29.30	338.53	28.94	338.53
	8	van Baarsen & Pu [46]	439.35	12721.40	521.57	16955.50	814.18	33730.60	*	*
		Piske et al. [38]	2639.26	12626.70	2781.08	14663.3	3361.54	22294.10	*	*
		Ours	366.60	6427.12	395.73	6427.12	373.99	6427.12	326.58	8551.02

Table 5: Comparisons of fuzzy PSI over low-dimensional sets for L_2 distance. "*" indicates unsupported or no implementation for the specific parameter setting. The best result is marked in blue.

Set size $m = n$	Dim. d	Protocol	Threshold $\delta = 16$		Threshold $\delta = 64$		Threshold $\delta = 256$		Threshold $\delta = 1024$	
			Time (s)	Comm. (MB)	Time (s)	Comm. (MB)	Time (s)	Comm. (MB)	Time (s)	Comm. (MB)
2^{12}	2	van Baarsen & Pu [46]	0.14	7.83	0.29	24.65	0.94	90.48	3.93	352.58
		Ours	0.59	2.88	0.57	3.32	0.58	3.32	0.57	3.85
	4	van Baarsen & Pu [46]	0.33	18.76	0.72	52.17	2.09	183.65	7.82	707.84
		Ours	0.60	6.54	0.73	8.13	0.74	8.13	0.74	9.88
	8	van Baarsen & Pu [46]	5.32	199.87	6.41	266.46	10.08	529.32	22.89	1578.13
		Ours	4.04	101.39	3.24	134.97	3.23	134.97	4.13	169.34
2^{16}	2	van Baarsen & Pu [46]	1.66	119.14	4.42	388.74	15.92	1444.29	68.34	5646.3
		Ours	3.12	27.19	3.65	34.17	3.77	34.17	3.86	42.44
	4	van Baarsen & Pu [46]	5.49	294.25	11.60	829.74	36.25	2937.84	162.54	11342.5
		Ours	5.21	85.85	6.83	111.34	6.93	111.34	7.61	139.33
	8	van Baarsen & Pu [46]	100.28	3197.70	123.78	4265.31	187.23	8479.44	423.36	25234.90
		Ours	73.01	1606.89	58.71	2145.39	65.18	2145.39	73.79	2696.60
2^{18}	2	van Baarsen & Pu [46]	6.99	475.64	19.69	1555.24	68.85	5781.38	298.37	22548.90
		Ours	13.35	105.38	14.42	133.38	14.95	133.38	15.51	166.64
	4	van Baarsen & Pu [46]	22.45	1176.75	47.77	3320.83	187.45	11761.30	*	*
		Ours	22.93	340.32	29.76	442.52	29.84	442.52	32.26	554.86
	8	van Baarsen & Pu [46]	482.04	12801.40	592.76	17075.50	898.91	33890.60	*	*
		Ours	365.36	6431.17	350.83	8587.77	306.63	8587.77	383.05	10795.40

OT-based fuzzy matching and follow the Cuckoo-hashing-based strategy of our fuzzy PSI for low-dimensional sets.

We further optimize the input domain used in the OT-based fuzzy matching phase. Our key observation is that for any points $\mathbf{q}_j$ and $\mathbf{w}_i$, having the same identifier $\text{id}_{\mathbf{q}_j} = \text{id}_{\mathbf{w}_i}$ implies $\text{dist}_{\infty}(\mathbf{w}_i, \mathbf{q}_j) \leq \delta$. Consequently, for every dimension $k \in [d]$, it holds that $|w_{i,k} - q_{j,k}| \leq \delta$. As shown in our OT-based fuzzy matching protocol in

Figure 15, the first step computes $t_k = w_{i,k} - q_{j,k}$, which lies in the range $[-\delta, \delta]$. Therefore, we can represent its secret shares using a smaller bit length. This optimization significantly reduces the overhead of subsequent operations whose complexity depends on the bit length of the secret sharing. We follow a strategy similar to our fuzzy PSI protocol for low-dimensional sets in Figure 11 to address false positives caused by the reduced input domain.

Table 6: Comparisons of fuzzy PSI over high-dimensional sets for L_1 distance. "*" indicates unsupported or no implementation for the specific parameter setting. The best result is marked in blue.

Set size $m = n$	Dim. d	Protocol	Threshold $\delta = 16$		Threshold $\delta = 64$		Threshold $\delta = 256$		Threshold $\delta = 1024$	
			Time (s)	Comm. (MB)	Time (s)	Comm. (MB)	Time (s)	Comm. (MB)	Time (s)	Comm. (MB)
2^{12}	16	van Baarsen & Pu [46]	0.46	49.49	1.68	181.35	7.29	705.9	26.95	2803.22
		Ours	2.93	39.32	3.18	47.43	2.98	55.62	3.37	63.75
	32	van Baarsen & Pu [46]	0.90	98.29	3.38	362.02	13.68	1411.69	54.73	5607.82
		Ours	5.29	74.79	5.94	91.09	5.66	107.58	5.96	123.90
	64	van Baarsen & Pu [46]	1.88	195.93	6.86	723.55	30.28	2823.77	115.75	11220.50
		Ours	9.49	145.93	10.15	178.39	10.31	211.37	11.36	244.06
2^{16}	16	van Baarsen & Pu [46]	7.99	786.94	28.99	2901.02	115.97	11311.50	*	*
		Ours	40.97	592.16	42.51	722.14	45.33	854.18	47.45	985.11
	32	van Baarsen & Pu [46]	16.13	1569.29	60.04	5797.39	252.21	22566.60	*	*
		Ours	81.10	1164.54	82.17	1424.65	86.62	1688.79	92.64	1950.79
	64	van Baarsen & Pu [46]	33.04	3134.60	126.01	11593.90	*	*	*	*
		Ours	158.10	2309.64	179.22	2830.14	179.47	3358.68	189.75	3883.10
2^{18}	16	van Baarsen & Pu [46]	35.17	3149.60	129.34	11614.90	*	*	*	*
		Ours	173.76	2364.69	182.71	2885.19	195.19	3413.73	225.93	3938.15
	32	van Baarsen & Pu [46]	70.20	6281.38	293.72	23152.10	*	*	*	*
		Ours	381.05	4659.33	399.78	5700.37	445.68	6758.63	438.60	7807.79
	64	van Baarsen & Pu [46]	142.61	12549.20	*	*	*	*	*	*
		Ours	974.50	9250.23	1023.99	11334.20	1069.14	13450.60	1118.85	15550.50

Table 7: Comparisons of fuzzy PSI over high-dimensional sets for L_2 distance. "*" indicates unsupported or no implementation for the specific parameter setting. The best result is marked in blue.

Set size $m = n$	Dim. d	Protocol	Threshold $\delta = 16$		Threshold $\delta = 64$		Threshold $\delta = 256$		Threshold $\delta = 1024$	
			Time (s)	Comm. (MB)	Time (s)	Comm. (MB)	Time (s)	Comm. (MB)	Time (s)	Comm. (MB)
2^{12}	16	van Baarsen & Pu [46]	0.48	51.74	1.71	184.72	6.76	710.40	28.63	2808.84
		Ours	1.51	50.35	1.68	58.47	1.76	66.66	1.97	74.79
	32	van Baarsen & Pu [46]	0.93	102.54	3.44	368.39	13.82	1420.19	57.93	5618.45
		Ours	2.43	97.25	2.65	113.55	2.86	130.04	3.23	146.36
	64	van Baarsen & Pu [46]	1.88	204.18	6.96	735.92	27.48	2840.27	116.44	11241.20
		Ours	4.23	191.26	4.72	223.72	5.04	256.70	5.83	289.39
2^{16}	16	van Baarsen & Pu [46]	8.14	822.94	30.03	2955.02	117.34	11383.50	*	*
		Ours	18.86	777.21	20.44	907.18	22.12	1039.23	25.85	1170.16
	32	van Baarsen & Pu [46]	16.55	1637.29	60.64	5899.39	263.34	22702.60	*	*
		Ours	33.28	1535.14	38.38	1795.25	41.36	2059.38	47.27	2321.38
	64	van Baarsen & Pu [46]	33.01	3266.60	126.18	11791.90	*	*	*	*
		Ours	72.69	3051.35	73.45	3571.85	82.49	4100.39	91.7	4624.81
2^{18}	16	van Baarsen & Pu [46]	35.68	3293.60	130.30	11830.90	*	*	*	*
		Ours	83.30	3111.16	92.05	3631.66	102.04	4160.21	106.12	4684.62
	32	van Baarsen & Pu [46]	77.92	6553.38	341.46	23560.10	*	*	*	*
		Ours	150.70	6152.83	168.78	7193.88	183.38	8252.13	214.89	9301.29

D Additional Experimental Results

We provide additional experimental results for L_p distance. Specifically, Tables 4 and 5 report the performance comparisons of fuzzy PSI over low-dimensional sets for L_1 and L_2 distance, respectively. Tables 6 and 7 report the performance comparisons of fuzzy PSI over high-dimensional sets for L_1 and L_2 distance, respectively.

E Security Proofs

E.1 Proof of Theorem 5.1

PROOF. We first establish the correctness of the protocol by analyzing the prefix matching and parameter bounds, and then construct simulators to demonstrate that the simulated execution is indistinguishable from the real-world protocol execution.

Correctness. We first consider the matching case, where it holds that $\text{dist}_\infty(\mathbf{q}, \mathbf{w}) \leq \delta$. This implies that $w_k \in [q_k - \delta, q_k + \delta]$ for all

dimensions $k \in [d]$. By Theorem 3.1, for each $k \in [d]$, there exists a unique index $t_k^* \in [u']$ such that $\tilde{w}_{k,t_k^*} \in \{\tilde{q}_{k,t}\}_{t \in [u]}$. By the correctness of the first $\mathcal{F}_{\text{OPPRF}}$, the receiver successfully learns $\hat{r}_{k,t_k^*} = r_k$, while the remaining evaluations $\{\hat{r}_{k,t}\}_{t \neq t_k^*}$ are independent and uniformly random. To ensure the second $\mathcal{F}_{\text{OPPRF}}$ is correctly programmed, all elements in $\{\hat{r}_{k,t}\}_{t \in [u']}$ must be distinct. As analyzed in the mismatch case below, setting ℓ_1 appropriately guarantees this distinctness with overwhelming probability. Consequently, by the correctness of the second $\mathcal{F}_{\text{OPPRF}}$, the sender learns $s_k = \hat{s}_k$ for all $k \in [d]$. This yields $s = \hat{s}$, and by the correctness of $\mathcal{F}_{\text{PEQT}}$, the receiver obtains the correct output 1.

We now consider the mismatch case where $\text{dist}_\infty(\mathbf{q}, \mathbf{w}) > \delta$. This implies there exists at least one dimension k^* such that $w_{k^*} \notin [q_{k^*} - \delta, q_{k^*} + \delta]$. By Theorem 3.1, the corresponding prefix sets are disjoint: $\{\tilde{q}_{k^*,t}\}_{t \in [u]} \cap \{\tilde{w}_{k^*,t}\}_{t \in [u']} = \emptyset$. Therefore, by the correctness of $\mathcal{F}_{\text{OPPRF}}$, all evaluations $\{\hat{r}_{k^*,t}\}_{t \in [u']}$ obtained by the receiver

are uniformly random and independent of uniformly random r_{k^*} . To ensure correctness, we must ensure two conditions: (1) no value in $\{\hat{r}_{k^*,t}\}_{t \in [u']}$ accidentally equals r_{k^*} to avoid false positives, and (2) all values in $\{\hat{r}_{k^*,t}\}_{t \in [u']}$ are distinct, preventing collisions when programming the second $\mathcal{F}_{\text{OPPRF}}$. In a single dimension, both requirements hold except with probability at most $(u' + 1)^2/2^{\ell_1}$. By setting the field size to $\ell_1 := \lambda + 2 \log(u' + 1) + \log d$, a union bound guarantees these conditions hold simultaneously across all d dimensions except with negligible probability in the security parameter λ . Under this parameterization, the evaluation query r_{k^*} is unprogrammed in the second $\mathcal{F}_{\text{OPPRF}}$, resulting in s_{k^*} being uniformly random. Because the dimensions are evaluated independently, the aggregate sum $s = \sum_{k \in [d]} s_k$ and $\hat{s} = \sum_{k \in [d]} \hat{s}_k$ are also uniformly random. Setting $\ell_2 := \lambda$ ensures the probability of an accidental collision $s = \hat{s}$ is bounded by $1/2^\lambda$, which is negligible. Thus, $\mathcal{F}_{\text{PEQT}}$ outputs 0, completing the correctness analysis.

Corrupted Sender. We construct a simulator Sim_S for a semi-honest corrupted sender. Given the sender's input q , Sim_S runs the adversary $\mathcal{A}$ and proceeds as follows:

- (1) *Simulating $\mathcal{F}_{\text{OPPRF}}$ (Step 2):* Sim_S receives the programmed points $\{(k||\tilde{q}_{k,t}, r_k)\}_{k \in [d], t \in [u]}$ generated by $\mathcal{A}$. Sim_S samples a random function F such that $F(k||\tilde{q}_{k,t}) = r_k$ for $k \in [d], t \in [u]$, and sends $\mathcal{O}^F$ to $\mathcal{A}$.
- (2) *Simulating $\mathcal{F}_{\text{OPPRF}}$ (Step 3):* After receiving $\{k||r_k\}_{k \in [d]}$ from $\mathcal{A}$, Sim_S samples $\{s_k\}_{k \in [d]} \leftarrow \mathbb{F}_{2^{\ell_2}}$, and sends them to $\mathcal{A}$.
- (3) *Simulating $\mathcal{F}_{\text{PEQT}}$ (Step 5):* Sim_S receives s from $\mathcal{A}$, and returns nothing.

We demonstrate the security of the simulation through a sequence of hybrid games:

- Hyb_0 : This is the real protocol execution.
- Hyb_1 : This hybrid behaves identically to Hyb_0 , except that instead of passing the honest receiver's input $\hat{s}$ to $\mathcal{F}_{\text{PEQT}}$ (Step 5), Sim_S receives s from $\mathcal{A}$ and returns nothing, which perfectly simulates the $\mathcal{F}_{\text{PEQT}}$'s behavior. Thus, the views in Hyb_0 and Hyb_1 are perfectly identical.
- Hyb_2 : This hybrid behaves identically to Hyb_1 , except that instead of passing the honest receiver's programmed points to $\mathcal{F}_{\text{OPPRF}}$ (Step 3), Sim_S receives $\mathcal{A}$'s queries $\{k||r_k\}_{k \in [d]}$ and returns uniformly random values $s_k \leftarrow \mathbb{F}_{2^{\ell_2}}$ for all $k \in [d]$ to $\mathcal{A}$. In Hyb_1 , s_k is either equal to the receiver's choice $\hat{s}_k$ (upon a match) or is a random function output. Because the honest receiver samples $\hat{s}_k \leftarrow \mathbb{F}_{2^{\ell_2}}$ uniformly at random, every s_k in Hyb_1 is perfectly uniform and independent of $\mathcal{A}$'s view. Thus, the views in Hyb_1 and Hyb_2 are perfectly identical.
- Hyb_3 : This hybrid behaves identically to Hyb_2 , except that instead of passing honest receiver's inputs $\{k||\tilde{w}_{k,t}\}_{k \in [d], t \in [u']}$ to $\mathcal{F}_{\text{OPPRF}}$ (Step 2), Sim_S receives the programmed points $\{(k||\tilde{q}_{k,t}, r_k)\}_{k \in [d], t \in [u]}$ from $\mathcal{A}$, samples a random function F such that $F(k||\tilde{q}_{k,t}) = r_k$ for $k \in [d], t \in [u]$, and sends $\mathcal{O}^F$ to $\mathcal{A}$. Since the real functionality also returns $\mathcal{O}^F$ according to these programmed points, the views in Hyb_2 and Hyb_3 are perfectly identical.

The final hybrid Hyb_3 is exactly the simulated execution of Sim_S . Therefore, the simulated view is perfectly indistinguishable from the real execution.

Corrupted Receiver. We construct a simulator Sim_R for a semi-honest corrupted receiver. Given the receiver's input w and the ideal output z , Sim_R runs $\mathcal{A}$ and proceeds as follows:

- (1) *Simulating $\mathcal{F}_{\text{OPPRF}}$ (Step 2):* After receiving the $\mathcal{A}$'s queries $\{k||\tilde{w}_{k,t}\}_{k \in [d], t \in [u']}$, Sim_R samples $\{\hat{r}_{k,t}\}_{k \in [d], t \in [u']} \leftarrow \mathbb{F}_{2^{\ell_1}}$, and returns them to $\mathcal{A}$.
- (2) *Simulating $\mathcal{F}_{\text{OPPRF}}$ (Step 3):* Sim_R receives the programmed points $\{(k||\hat{r}_{k,t}, \hat{s}_k)\}_{k \in [d], t \in [u']}$ from $\mathcal{A}$. Sim_R samples a random function F' such that $F'(k||\hat{r}_{k,t}) = \hat{s}_k$ for $k \in [d], t \in [u']$ and returns $\mathcal{O}^{F'}$ to $\mathcal{A}$.
- (3) *Simulating $\mathcal{F}_{\text{PEQT}}$ (Step 5):* Sim_R receives the aggregate sum $\hat{s}$ from $\mathcal{A}$, and returns the ideal output z to $\mathcal{A}$.

We demonstrate the security of the simulation through a sequence of hybrid games:

- Hyb_0 : This is the real protocol execution.
- Hyb_1 : This hybrid behaves identically to Hyb_0 , except that instead of passing the honest sender's input s to $\mathcal{F}_{\text{PEQT}}$ (Step 5), Sim_R receives $\mathcal{A}$'s aggregate sum $\hat{s}$ and directly returns the ideal output z to $\mathcal{A}$. The only difference between Hyb_0 and Hyb_1 occurs if the protocol produces an incorrect evaluation. As established in the correctness analysis, the probability is statistically negligible. Thus, the views in Hyb_0 and Hyb_1 are statistically indistinguishable.
- Hyb_2 : This hybrid behaves identically to Hyb_1 , except that instead of passing the honest sender's queries to $\mathcal{F}_{\text{OPPRF}}$ (Step 3), Sim_R receives the programmed points $\{(k||\hat{r}_{k,t}, \hat{s}_k)\}_{k \in [d], t \in [u']}$ from $\mathcal{A}$, samples a random function F' such that $F'(k||\hat{r}_{k,t}) = \hat{s}_k$ for $k \in [d], t \in [u']$ and returns $\mathcal{O}^{F'}$ to $\mathcal{A}$. Since Hyb_1 also returns $\mathcal{O}^{F'}$ according to these programmed points, the views in Hyb_1 and Hyb_2 are perfectly identical.
- Hyb_3 : This hybrid behaves identically to Hyb_2 , except that instead of passing the honest sender's programmed points to $\mathcal{F}_{\text{OPPRF}}$ (Step 2), Sim_R receives $\mathcal{A}$'s queries $\{k||\tilde{w}_{k,t}\}_{k \in [d], t \in [u']}$ and returns uniformly random values $\{\hat{r}_{k,t}\}_{k \in [d], t \in [u']} \leftarrow \mathbb{F}_{2^{\ell_1}}$ to $\mathcal{A}$. In Hyb_2 , $\mathcal{A}$ receives either the sender's payload r_k or a random function evaluation. Because the honest sender samples $\{r_k\}_{k \in [d]} \leftarrow \mathbb{F}_{2^{\ell_1}}$ uniformly at random, every returned evaluation is uniformly distributed. Thus, the views in Hyb_2 and Hyb_3 are perfectly identical.

The final hybrid Hyb_3 is exactly the simulated execution of Sim_R . Therefore, the simulated view is statistically indistinguishable from the real execution. $\square$

E.2 Proof of Theorem 5.2

PROOF. We first establish the correctness of the protocol, and then construct simulators to demonstrate that the simulated execution is perfectly indistinguishable from the real-world protocol execution.

Correctness. The correctness of the protocol relies on accurately extracting the Most Significant Bit (MSB) of the terms $\delta - (q_k - w_k)$ and $\delta + (q_k - w_k)$. Assuming the inputs q_k, w_k , and the threshold δ are strictly bounded by z , the values of these two terms fall within the interval $(-z, 2z)$. To prevent wrap-around in the two's complement representation, the ring $\mathbb{Z}_{2^t}$ must be parameterized

such that $\ell \geq \log z + 2$. Under this condition, these MSBs are correctly evaluated, and the overall correctness of the protocol directly follows from the correctness of the underlying $\mathcal{F}_{\text{MSB}}$ and $\mathcal{F}_{\text{AND}}$ functionalities.

Corrupted Sender. We construct a simulator Sim_S for a semi-honest corrupted sender. Given the sender's input q , Sim_S operates by running the adversary $\mathcal{A}$ and proceeds as follows:

- (1) *Simulating $\mathcal{F}_{\text{MSB}}$ (Step 2):* After receiving $\delta - [t_k]_S^\ell$ from $\mathcal{A}$, Sim_S samples uniformly random shares $[a_k]_S^1 \leftarrow \mathbb{Z}_2$ for all $k \in [d]$, and sends them to $\mathcal{A}$.
- (2) *Simulating $\mathcal{F}_{\text{MSB}}$ (Step 3):* After receiving $\delta + [t_k]_S^\ell$ from $\mathcal{A}$, Sim_S samples uniformly random shares $[b_k]_S^1 \leftarrow \mathbb{Z}_2$ for all $k \in [d]$, and sends them to $\mathcal{A}$.
- (3) *Simulating $\mathcal{F}_{\text{AND}}$ (Step 4):* After receiving $\mathcal{A}$'s inputs, Sim_S samples a random share $[z]_S^1 \leftarrow \mathbb{Z}_2$ and sends it to $\mathcal{A}$.

We argue the simulated view of $\mathcal{A}$ is identically distributed to the real execution. Since the protocol operates in the $(\mathcal{F}_{\text{MSB}}, \mathcal{F}_{\text{AND}})$ -hybrid model, the messages received by $\mathcal{A}$ consist solely of its own secret shares. In the real execution, the shares $\{[a_k]_S^1\}_{k \in [d]}$, $\{[b_k]_S^1\}_{k \in [d]}$, and $[z]_S^1$ are generated as uniformly random secret shares of the underlying functionality outputs. In the simulated execution, Sim_S explicitly samples them from the uniform distribution over $\mathbb{Z}_2$. Thus, the simulated view of $\mathcal{A}$ is identically distributed to its view in the real protocol.

Corrupted Receiver. We then construct a simulator Sim_R for a semi-honest corrupted receiver. Given the receiver's input w and the ideal output z from $\mathcal{F}_{\text{Match}}$, Sim_R runs $\mathcal{A}$ and proceeds as follows:

- (1) *Simulating $\mathcal{F}_{\text{MSB}}$ (Step 2):* After receiving $\delta - [t_k]_R^\ell$ from $\mathcal{A}$, Sim_R samples uniform random shares $[a_k]_R^1 \leftarrow \mathbb{Z}_2$ for all $k \in [d]$ and sends them to $\mathcal{A}$.
- (2) *Simulating $\mathcal{F}_{\text{MSB}}$ (Step 3):* After receiving $\delta + [t_k]_R^\ell$ from $\mathcal{A}$, Sim_R samples uniform random shares $[b_k]_R^1 \leftarrow \mathbb{Z}_2$ for all $k \in [d]$ and sends them to $\mathcal{A}$.
- (3) *Simulating $\mathcal{F}_{\text{AND}}$ (Step 4):* After receiving $\mathcal{A}$'s inputs, Sim_R samples a random share $[z]_R^1 \leftarrow \mathbb{Z}_2$ and sends it to $\mathcal{A}$.
- (4) *Simulating the final message (Step 5):* Sim_R computes the simulated sender's message as $[z]_S^1 := [z]_R^1 \oplus z \in \mathbb{Z}_2$, and sends it to $\mathcal{A}$.

We argue the simulated view of $\mathcal{A}$ is identically distributed to the real execution. Similar to the corrupted sender case, the shares $\{[a_k]_R^1\}_{k \in [d]}$, $\{[b_k]_R^1\}_{k \in [d]}$, and $[z]_R^1$ are uniformly random in both the real and simulated executions. The only constraint on the view is the final message $[z]_S^1$. In the real protocol, $[z]_S^1$ is the honest sender's share, which satisfies $[z]_S^1 \oplus [z]_R^1 = z$. In the simulation, Sim_R explicitly programs $[z]_S^1$ to satisfy this exact relation using the ideal output z . Therefore, the simulated view of $\mathcal{A}$ and the real protocol view are perfectly identically distributed. Thus, the proof is complete. $\square$

E.3 Proof of Theorem B.1

PROOF. We first establish the correctness of the protocol, and then construct simulators to demonstrate that the simulated execution is perfectly indistinguishable from the real-world protocol execution.

Correctness. The correctness of the protocol relies on accurately extracting the Most Significant Bit (MSB) of the term $f := \delta^p - \sum_{k \in [d]} f_k$, where $f_k := |q_k - w_k|^p$. Assuming the inputs q_k, w_k and the threshold δ are bounded by z , the sum f falls within the interval $(-d \cdot z^p, z^p)$. To prevent wrap-around in the ring $\mathbb{Z}_{2^\ell}$, the ring must be parameterized such that $\ell \geq \log_2(d \cdot z^p) + 1$. Under this condition, the MSB accurately evaluates whether the distance exceeds the threshold, and the overall correctness directly follows from the correctness of the underlying $\mathcal{F}_{\text{ABS}}$, $\mathcal{F}_{\text{Mult}}$, and $\mathcal{F}_{\text{MSB}}$ functionalities.

Corrupted Sender. We construct a simulator Sim_S for a semi-honest corrupted sender. Given the sender's input q , Sim_S operates by running the adversary $\mathcal{A}$ and proceeds as follows:

- (1) *Simulating $\mathcal{F}_{\text{ABS}}$ or $\mathcal{F}_{\text{Mult}}$ (Step 2):* After receiving $[t_k]_S^\ell$ from $\mathcal{A}$, Sim_S samples uniform random shares $[f_k]_S^\ell \leftarrow \mathbb{Z}_{2^\ell}$ each all $k \in [d]$ and sends them to $\mathcal{A}$.
- (2) *Simulating $\mathcal{F}_{\text{MSB}}$ (Step 3):* After receiving $[f]_S^\ell$ from $\mathcal{A}$, Sim_S samples a uniform random share $[z]_S^1 \leftarrow \mathbb{Z}_2$ and sends it to $\mathcal{A}$.

We argue the simulated view of $\mathcal{A}$ is identically distributed to the real execution. In the $(\mathcal{F}_{\text{ABS}}, \mathcal{F}_{\text{Mult}}, \mathcal{F}_{\text{MSB}})$ -hybrid model, all intermediate values received by $\mathcal{A}$ are purely additive secret shares. In the real protocol, the shares $\{[f_k]_S^\ell\}_{k \in [d]}$ and $[z]_S^1$ are uniformly distributed. In the simulation, Sim_S explicitly samples them from the uniform distributions over $\mathbb{Z}_{2^\ell}$ and $\mathbb{Z}_2$, respectively. Thus, the simulated view of $\mathcal{A}$ is identically distributed to its view in the real protocol.

Corrupted Receiver. We construct a simulator Sim_R for a semi-honest corrupted receiver. Given the receiver's input w and the ideal output $z \in \mathbb{Z}_2$ from $\mathcal{F}_{\text{Match}}$, Sim_R runs $\mathcal{A}$ and proceeds as follows:

- (1) *Simulating $\mathcal{F}_{\text{ABS}}$ or $\mathcal{F}_{\text{Mult}}$ (Step 2):* After receiving $[t_k]_R^\ell$ from $\mathcal{A}$, Sim_R samples uniform random shares $[f_k]_R^\ell \leftarrow \mathbb{Z}_{2^\ell}$ for each $k \in [d]$ and sends them to $\mathcal{A}$.
- (2) *Simulating $\mathcal{F}_{\text{MSB}}$ (Step 3):* After receiving $[f]_R^\ell$ from $\mathcal{A}$, Sim_R samples a uniform random share $[z]_R^1 \leftarrow \mathbb{Z}_2$ and sends it to $\mathcal{A}$.
- (3) *Simulating the final message (Step 4):* Sim_R computes the message as $m_S := z \oplus [z]_R^1 \in \mathbb{Z}_2$. Sim_R then sends m_S to $\mathcal{A}$, simulating the message $(1 \oplus [z]_S^1)$ from the honest sender.

We argue the simulated view of $\mathcal{A}$ is identically distributed to the real execution. Similar to the corrupted sender case, the shares $\{[f_k]_R^\ell\}_{k \in [d]}$ and $[z]_R^1$ are uniformly random in both executions. The final message m_S in the real protocol is $1 \oplus [z]_S^1$, which perfectly satisfies the output relation $z = m_S \oplus [z]_R^1$. In the simulation, Sim_R directly constructs m_S to satisfy this exact relation using the ideal output z . Therefore, the simulated view of $\mathcal{A}$ and the real protocol view are perfectly identically distributed. Thus, the proof is complete. $\square$

E.4 Proof of Theorem 6.1

PROOF. We first analyze the protocol's correctness and then prove the security of the protocol by constructing simulators for both a corrupted sender and a corrupted receiver.

Correctness. We first consider the matching case, where for a sender point $q_j \in Q$, there exists a receiver point $w_i \in W$ such

that $\text{dist}_p(\mathbf{w}_i, \mathbf{q}_j) \leq \delta$. By the definition of spatial hashing, there must exist a unique index $h^* \in [2^d]$ such that the computed cell identifiers match: $C_{\mathbf{w}_i, h^*} = C_{\mathbf{q}_j}$. By the unique center assumption, the sender's points are sufficiently separated such that each grid cell contains at most one sender point. Consequently, the Cuckoo hash table T_q is successfully constructed with overwhelming probability. Let j^* denote the bin index of the tuple $(C_{\mathbf{q}_j}, \mathbf{q}_j)$ in T_q . Because both parties use identical hash functions, the receiver's corresponding tuple $(C_{\mathbf{w}_i, h^*}, \mathbf{w}_i)$ is mapped to the same bin index $T_w[j^*]$. By the unique ball assumption, a cell intersects at most one δ -radius receiver ball. Thus, the cell identifiers $C_{\mathbf{w}_i, h}$ are unique for all $i \in [n], h \in [2^d]$, ensuring no key collisions exist within the programmed sets $\{L_j\}_{j \in [m']}$ provided to $\mathcal{F}_{\text{OPPRF}}$. By the correctness of $\mathcal{F}_{\text{OPPRF}}$, the sender queries with $(C_{\mathbf{q}_j} \parallel \tau)$ from $T_q[j^*]$ and successfully learns the programmed payload:

$$\hat{s}_j^* \parallel \hat{\mathbf{r}}_j^* = s_{j^*} \parallel (\text{shift}_{\lceil \log 6\delta \rceil}(\mathbf{w}_i, C_{\mathbf{w}_i, h^*} - 2\delta) - \mathbf{r}_{j^*} \bmod 2^{\ell_2}).$$

Meanwhile, the receiver holds s_{j^*} and $\mathbf{r}_{j^*}$ for the j^* -th bin of T_w .

To ensure that the computation will not overflow in OT-based fuzzy matching, we set the bit length ℓ_2 according to the analysis in Appendix E.2 and Appendix E.3. Given that the input is less than 6δ in the reduced domain, we set $\ell_2 := \lceil \log(6\delta) \rceil + 2$ for L_∞ distance and $\ell_2 := \lceil \log d(6\delta)^p \rceil + 1$ for L_p distance. Note that for L_2 distance, ℓ_2 can be further optimized to $\lceil \log d(3\delta)^2 \rceil$, according to the fact that if the cell of $\mathbf{q}_j$ and $\mathbf{w}_i$'s ball is matched then $|q_{j,k} - w_{i,k}| < 3\delta$ for all $k \in [d]$. Finally, by the correctness of the functionalities $\mathcal{F}_{\text{ssFMatch}}$, $\mathcal{F}_{\text{ssPEQT}}$, $\mathcal{F}_{\text{AND}}$, $\mathcal{F}_{\text{Perm}}$, and $\mathcal{F}_{\text{OT}}$, the receiver $\mathcal{R}$ correctly obtains $\mathbf{q}_j$.

We next consider the mismatch case, where for a sender point $\mathbf{q}_j \in Q$, it holds that $\text{dist}_p(\mathbf{w}_i, \mathbf{q}_j) > \delta$ for all $\mathbf{w}_i \in W$. Let j^* denote its bin index in T_q . We distinguish two sub-cases: *Case 1 (Cell Collision, Distance Mismatch)*: There exists a $\mathbf{w}_i \in W$ and $h^* \in [2^d]$ such that $C_{\mathbf{w}_i, h^*} = C_{\mathbf{q}_j}$. Because the distance strictly exceeds δ , the correctness of $\mathcal{F}_{\text{ssFMatch}}$ ensures the reconstructed output is $v_{j^*} = 0$. *Case 2 (No Cell Collision)*: For all $\mathbf{w}_i \in W$ and $h \in [2^d]$, $C_{\mathbf{w}_i, h} \neq C_{\mathbf{q}_j}$. The sender's query to $\mathcal{F}_{\text{OPPRF}}$ is unprogrammed, yielding a uniformly random value $\hat{s}_{j^*} \in \mathbb{F}_{2^{\ell_1}}$. The probability of an accidental collision $\hat{s}_{j^*} = s_{j^*}$ is $2^{-\ell_1}$. Setting $\ell_1 := \lambda + \log m'$ ensures that by a union bound, the probability of any such collision across all m' bins is bounded by $2^{-\lambda}$. Thus, except with negligible probability, $\mathcal{F}_{\text{ssPEQT}}$ correctly ensures the reconstructed output is $\mu_{j^*} = 0$. In both sub-cases, the $\mathcal{F}_{\text{AND}}$ functionality produces shares of 0, and the final $\mathcal{F}_{\text{OT}}$ delivers $\perp$ to the receiver. This completes the correctness proof.

Corrupted Sender. We construct a simulator Sim_S for a semi-honest corrupted sender. Given the sender's input Q , Sim_S runs the adversary $\mathcal{A}$ and proceeds as follows:

- (1) *Simulating $\mathcal{F}_{\text{OPPRF}}$ (Step 5)*: After receiving the queries, Sim_S samples uniformly random values $\hat{s}_j \leftarrow \mathbb{F}_{2^{\ell_1}}$ and $\hat{\mathbf{r}}_j \leftarrow \mathbb{Z}_{2^{\ell_2}}^d$ for all $j \in [m']$, and returns $\{\hat{s}_j \parallel \hat{\mathbf{r}}_j\}_{j \in [m']}$ to $\mathcal{A}$.
- (2) *Simulating $\mathcal{F}_{\text{ssFMatch}}$, $\mathcal{F}_{\text{ssPEQT}}$, and $\mathcal{F}_{\text{AND}}$ (Steps 6-8)*: After receiving $\mathcal{A}$'s inputs to these functionalities, Sim_S simulates the outputs by sampling uniformly random bits $v_j^S, \mu_j^S, b_j^S \leftarrow \mathbb{F}_2$ for all $j \in [m']$ and returns them to $\mathcal{A}$.

- (3) *Simulating $\mathcal{F}_{\text{Perm}}$ (Step 9)*: After receiving $\{b_j^S\}_{j \in [m']}$ from $\mathcal{A}$, Sim_S samples a random permutation π over $[m']$ and returns it to $\mathcal{A}$.
- (4) *Simulating $\mathcal{F}_{\text{OT}}$ (Step 10)*: After receiving OT's inputs from $\mathcal{A}$, Sim_S provides no output to $\mathcal{A}$.

We demonstrate the security of the simulation through a sequence of hybrid games:

- **Hyb₀**: The real protocol execution in the hybrid model.
- **Hyb₁**: This hybrid behaves identically to **Hyb₀**, except that instead of invoking $\mathcal{F}_{\text{Perm}}$ (Step 9) and $\mathcal{F}_{\text{OT}}$ (Step 10), Sim_S returns a random permutation π over $[m']$ as the output of $\mathcal{F}_{\text{Perm}}$, and provides no output for $\mathcal{A}$ in $\mathcal{F}_{\text{OT}}$, which perfectly simulates the functionalities' behavior. Thus, the views in **Hyb₀** and **Hyb₁** are perfectly identical.
- **Hyb₂**: This hybrid behaves identically to **Hyb₁** except the simulation of $\mathcal{F}_{\text{ssFMatch}}$, $\mathcal{F}_{\text{ssPEQT}}$, and $\mathcal{F}_{\text{AND}}$ (Steps 6-8). Sim_S receives $\mathcal{A}$'s inputs and simply returns uniformly random bits $v_j^S, \mu_j^S, b_j^S \leftarrow \mathbb{F}_2$ for all $j \in [m']$. Because these ideal functionalities output uniform secret shares to $\mathcal{A}$ in **Hyb₁**, the views in **Hyb₁** and **Hyb₂** are perfectly identical.
- **Hyb₃**: This hybrid behaves identically to **Hyb₂**, except that instead of passing the honest receiver's programmed points to $\mathcal{F}_{\text{OPPRF}}$ (Step 5), Sim_R receives $\mathcal{A}$'s queries and returns uniformly random values $\hat{s}_j \leftarrow \mathbb{F}_{2^{\ell_1}}$ and $\hat{\mathbf{r}}_j \leftarrow \mathbb{Z}_{2^{\ell_2}}^d$ for all $j \in [m']$. In **Hyb₂**, if a valid spatial match occurs, $\hat{\mathbf{r}}_j$ is masked by the receiver's uniformly random value $\mathbf{r}_j$, making it uniformly distributed. If no match occurs, it is a random function evaluation. The case of $\hat{s}_j$ is similar. Because the honest receiver samples all s_j and $\mathbf{r}_j$ uniformly at random, every evaluation $\mathcal{A}$ receives in **Hyb₂** is perfectly uniform and independent of $\mathcal{A}$'s input. Thus, the views in **Hyb₂** and **Hyb₃** are perfectly identical.

The final hybrid **Hyb₃** is exactly the simulated execution of Sim_S . Therefore, the simulated view is perfectly indistinguishable from the real execution.

Corrupted Receiver. We construct a simulator Sim_R for a semi-honest corrupted receiver. Given the receiver's input W and the ideal output set Z from $\mathcal{F}_{\text{PSI}}$, Sim_R runs $\mathcal{A}$ and proceeds as follows:

- (1) *Simulating $\mathcal{F}_{\text{OPPRF}}$ (Step 5)*: After receiving the programmed points $\{L_j\}_{j \in [m']}$ from $\mathcal{A}$, Sim_R samples a random function F programmed on $\{L_j\}_{j \in [m']}$. Sim_R sends $\mathcal{O}^F$ to $\mathcal{A}$.
- (2) *Simulating $\mathcal{F}_{\text{ssFMatch}}$, $\mathcal{F}_{\text{ssPEQT}}$, and $\mathcal{F}_{\text{AND}}$ (Steps 6-8)*: After receiving $\mathcal{A}$'s inputs, Sim_R samples uniformly random bits $v_j^R, \mu_j^R, b_j^R \leftarrow \mathbb{F}_2$ for all $j \in [m']$ and returns them to $\mathcal{A}$.
- (3) *Simulating $\mathcal{F}_{\text{Perm}}$ and $\mathcal{F}_{\text{OT}}$ (Steps 9-10)*: To simulate the final delivery of the intersection, Sim_R pads the ideal output set Z with $\perp$ elements until it contains exactly m' items. Sim_R shuffles this padded set randomly to yield $\{z_j\}_{j \in [m']}$. For each $j \in [m']$, Sim_R defines $\hat{b}_j := 0$ if $z_j = \perp$, and $\hat{b}_j := 1$ otherwise. After receiving $\mathcal{A}$'s inputs, Sim_R returns $\{\hat{b}_j\}_{j \in [m']}$ as the output of $\mathcal{F}_{\text{Perm}}$ and $\{z_j\}_{j \in [m']}$ as the output of the final $\mathcal{F}_{\text{OT}}$.

We demonstrate the security of the simulation through a sequence of hybrid games:

- **Hyb₀**: This is the real protocol execution.

- **Hyb₁**: This hybrid behaves identically to Hyb₀ except the simulation of $\mathcal{F}_{\text{Perm}}$ and $\mathcal{F}_{\text{OT}}$ (Steps 9-10). $\text{Sim}_{\mathcal{R}}$ pads the ideal output set Z with $\perp$ elements until it contains exactly m' items. It then randomly shuffles this padded set to yield $\{z_j\}_{j \in [m']}$. For each $j \in [m']$, $\text{Sim}_{\mathcal{R}}$ defines $\hat{b}_j := 1$ if $z_j \neq \perp$, and $\hat{b}_j := 0$ otherwise. It returns $\{\hat{b}_j\}_{j \in [m']}$ as the output of $\mathcal{F}_{\text{Perm}}$ and $\{z_j\}_{j \in [m']}$ as the output of $\mathcal{F}_{\text{OT}}$. In Hyb₀, $\mathcal{F}_{\text{Perm}}$ applies a uniformly random permutation to the underlying match bits, and $\mathcal{F}_{\text{OT}}$ returns the corresponding intersection elements interspersed with $\perp$ s. Because the permutation is uniformly random and completely hidden from $\mathcal{A}$, the sequence of bits and retrieved payloads is simply a uniformly random arrangement of exactly $|Z|$ matches and $m' - |Z|$ $\perp$ s. The only difference between Hyb₀ and Hyb₁ occurs if the protocol produces an incorrect evaluation. As established in the correctness analysis, the probability is statistically negligible. Thus, the views in Hyb₀ and Hyb₁ are statistically indistinguishable.
- **Hyb₂**: This hybrid behaves identically to Hyb₁ except the simulation of $\mathcal{F}_{\text{ssFMatch}}$, $\mathcal{F}_{\text{ssPEQT}}$, and $\mathcal{F}_{\text{AND}}$ (Steps 6-8). $\text{Sim}_{\mathcal{R}}$ receives $\mathcal{A}$'s inputs to these functionalities and returns uniformly random bits $v_j^{\mathcal{R}}, \mu_j^{\mathcal{R}}, b_j^{\mathcal{R}} \leftarrow \mathbb{F}_2$ for all $j \in [m']$. In Steps 6-8, the intermediate values $v_j^{\mathcal{R}}, \mu_j^{\mathcal{R}}, b_j^{\mathcal{R}}$ are uniform secret shares in Hyb₁, which $\text{Sim}_{\mathcal{R}}$ correctly models by sampling uniformly. Thus, the views in Hyb₁ and Hyb₂ are perfectly identical.
- **Hyb₃**: This hybrid behaves identically to Hyb₂ except the simulation of $\mathcal{F}_{\text{OPPRF}}$ (Step 5). After receiving the programmed points $\{L_j\}_{j \in [m']}$ from $\mathcal{A}$, $\text{Sim}_{\mathcal{R}}$ samples a random function F programmed on $\{L_j\}_{j \in [m']}$. $\text{Sim}_{\mathcal{R}}$ sends O^F to $\mathcal{A}$. Since Hyb₂ also returns O^F according to these programmed points, the views in Hyb₂ and Hyb₃ are perfectly identical.

The final hybrid Hyb₃ is exactly the simulated execution of $\text{Sim}_{\mathcal{R}}$. Therefore, the simulated view is statistically indistinguishable from the real execution. $\square$

E.5 Proof of Theorem 7.2

PROOF. We first analyze the protocol's correctness and then prove the security of the protocol by constructing simulators for both a corrupted sender and a corrupted receiver.

Correctness. Before analyzing correctness for different cases, we illustrate the distinctness of programmed points in two invocations of OPPRF. Below, we consider any dimension $k \in [d]$. (1) *In the first OPPRF invocation*, the globally disjoint assumption of the sender set ensures that for any $j, j' \in [m]$, $[q_{j,k} - \delta, q_{j,k} + \delta] \cap [q_{j',k} - \delta, q_{j',k} + \delta] = \emptyset$ and hence $|q_{j,k} - q_{j',k}| > 2\delta$. By Corollary 3.2, this implies the intersection of their prefixes is empty: $\{\tilde{q}_{j,k,t}\}_{t \in [u]} \cap \{\tilde{q}_{j',k,t}\}_{t \in [u]} = \emptyset$. Therefore, all programmed points $\{\tilde{q}_{j,k,t}\}_{j \in [m], t \in [u]}$ of the first OPPRF are distinct. (2) *In the second OPPRF invocation*, the globally disjoint assumption of the receiver set ensures that for any $i, i' \in [n]$, $[w_{i,k} - \delta, w_{i,k} + \delta] \cap [w_{i',k} - \delta, w_{i',k} + \delta] = \emptyset$ and hence $|w_{i,k} - w_{i',k}| > 2\delta$. By Corollary 3.3, this implies the intersection of their prefixes is empty: $\{\tilde{w}_{i,k,t}\}_{t \in [u']} \cap \{\tilde{w}_{i',k,t}\}_{t \in [u']} = \emptyset$. Moreover, by Corollary 3.4, for each sender prefix set $\{\tilde{q}_{j,k,t}\}_{t \in [u]}$, there is at most one receiver

prefix set $\{\tilde{w}_{i,k,t}\}_{t \in [u']}$ with a unique t intersected with it. Therefore, by the correctness of the first OPPRF, $\{\hat{r}_{i,k,t}\}_{i \in [n], t \in [u']}$ are independent and uniformly random, since $\hat{r}_{i,k,t}$ is either a random function value (unmatched with all programmed points) or some $r_{j,k}$ (matched with a point). By setting appropriate bit lengths as analyzed below, all values within the set $\{\hat{r}_{i,k,t}\}_{i \in [n], t \in [u']}$ for a given dimension k are distinct. This ensures that all programmed points of the second OPPRF are distinct.

With the correct invocations of OPPRF established, we proceed to analyze the correctness for different cases.

We first consider the matching case where for a sender point $q_j \in Q$, there exists $w_i \in W$ such that $\text{dist}_{\infty}(w_i, q_j) \leq \delta$. This implies that $w_{i,k} \in [q_{j,k} - \delta, q_{j,k} + \delta]$ for all dimensions $k \in [d]$. By Theorem 3.1, for each dimension $k \in [d]$, there exists a unique index $t_k^* \in [u']$ such that $\tilde{w}_{i,k,t_k^*} \in \{\tilde{q}_{j,k,t}\}_{t \in [u]}$. By the correctness of the first $\mathcal{F}_{\text{OPPRF}}$, the receiver successfully learns $\hat{r}_{i,k,t_k^*} = r_{j,k}$. Similarly, by the correctness of the second $\mathcal{F}_{\text{OPPRF}}$, the sender learns $s_{j,k} = \hat{s}_{i,k}$. This results in the aggregate sums matching $s_j = \hat{s}_i$. Finally, by the correctness of $\mathcal{F}_{\text{WLPSI}}$, the receiver queries $\hat{s}_i$, and successfully obtains s_j and the associated payload q_j .

We next consider the mismatch case, where for a sender point $q_j \in Q$, it holds that $\text{dist}_{\infty}(w_i, q_j) > \delta$ for all $w_i \in W$. This yields that for any w_i , there exists at least one dimension k^* such that $w_{i,k^*} \notin [q_{j,k^*} - \delta, q_{j,k^*} + \delta]$. By Theorem 3.1, this implies that the intersection of their prefixes is empty: $\{\tilde{q}_{j,k^*,t}\}_{t \in [u]} \cap \{\tilde{w}_{i,k^*,t}\}_{t \in [u']} = \emptyset$. By the correctness of the first $\mathcal{F}_{\text{OPPRF}}$, all evaluations $\{\hat{r}_{i,k^*,t}\}_{t \in [u']}$ obtained by the receiver are uniformly random and are independent of r_{j,k^*} . To ensure correctness of the entire protocol, we must guarantee two conditions: (1) no random function evaluation $\{\hat{r}_{i,k^*,t}\}_{t \in [u']}$ accidentally collides with any of the sender's payloads r_{j,k^*} (preventing false positives), and (2) all values within the set $\{\hat{r}_{i,k^*,t}\}_{i \in [n], t \in [u']}$ are strictly distinct, ensuring the receiver does not provide duplicate keys when programming the second $\mathcal{F}_{\text{OPPRF}}$. Considering the execution of OPPRF on the two sets, the total number of points drawn in each dimension is bounded by $(nu' + m)$. The probability of any collision occurring among these points is bounded by $(nu' + m)^2/2^{\ell_1}$. Considering all d dimensions, setting $\ell_1 := \lambda + 2 \log(nu' + m) + \log d$ ensures the probability of a collision is bounded by $1/2^{\lambda}$, which is negligible. Therefore, $r_{j,k^*} \notin \{\hat{r}_{i,k^*,t}\}_{t \in [u']}$ with overwhelming probability.

Under the above condition, by the correctness of the second $\mathcal{F}_{\text{OPPRF}}$, the sender receives a uniformly random s_{j,k^*} , independent to $\hat{s}_{i,k^*}$, since s_{j,k^*} is either a random value (unmatched with all programmed points) or some other $\hat{s}_{i',k^*}$ (matched with a point from $\hat{r}_{i',k^*,t}$). Because dimensions are evaluated independently, and because for each $i \in [n]$, there exists k^* such that s_{j,k^*} and $\hat{s}_{i,k^*}$ are uniformly random and independent, the aggregate sums s_j and $\hat{s}_i$ are uniformly random and independent. Setting $\ell_2 := \lambda + \log m + \log n$ guarantees that the probability of an accidental collision $s_j = \hat{s}_i$ across all $m \times n$ possible pairs is bounded by $(mn)/2^{\ell_2} \leq 1/2^{\lambda}$. Thus, the $\mathcal{F}_{\text{WLPSI}}$ query fails, returning nothing for this mismatch, completing the correctness proof.

Corrupted Sender. We construct a simulator $\text{Sim}_{\mathcal{S}}$ for a semi-honest corrupted sender. Given the sender's input Q , $\text{Sim}_{\mathcal{S}}$ runs the adversary $\mathcal{A}$ and proceeds as follows:

- (1) *Simulating $\mathcal{F}_{\text{OPPRF}}$ (Step 3)*: Sim_S receives the programmed points $\{(k\|\tilde{q}_{j,k,t}, r_{j,k})\}_{j \in [m], k \in [d], t \in [u]}$ generated by $\mathcal{A}$. Sim_S samples a random function F such that $F(k\|\tilde{q}_{j,k,t}) = r_{j,k}$ for $j \in [m], k \in [d], t \in [u]$, and returns O^F to $\mathcal{A}$.
- (2) *Simulating $\mathcal{F}_{\text{OPPRF}}$ (Step 4)*: After receiving the $\mathcal{A}$'s queries $\{k\|r_{j,k}\}_{j \in [m], k \in [d]}$, Sim_S samples uniformly random values $s_{j,k} \leftarrow \mathbb{F}_{2^{\ell_2}}$ for all $j \in [m], k \in [d]$, and sends them to $\mathcal{A}$.
- (3) *Simulating $\mathcal{F}_{\text{WLPSI}}$ (Step 6)*: After receiving $\{(s_j, \mathbf{q}_j)\}_{j \in [m]}$, Sim_S returns nothing.

We demonstrate the security of the simulation through a sequence of hybrid games:

- Hyb_0 : This is the real protocol execution.
- Hyb_1 : This hybrid behaves identically to Hyb_0 , except that instead of invoking $\mathcal{F}_{\text{WLPSI}}$ (Step 6), Sim_S receives $\{(s_j, \mathbf{q}_j)\}_{j \in [m]}$ from $\mathcal{A}$ and returns nothing, which perfectly simulates the $\mathcal{F}_{\text{WLPSI}}$'s behavior. Thus, the views in Hyb_0 and Hyb_1 are perfectly identical.
- Hyb_2 : This hybrid behaves identically to Hyb_1 , except that instead of passing the honest receiver's programmed points to $\mathcal{F}_{\text{OPPRF}}$ (Step 4), Sim_S directly returns uniformly random values $\{s_{j,k}\}_{j \in [m], k \in [d]} \leftarrow \mathbb{F}_{2^{\ell_2}}$ to $\mathcal{A}$. In Hyb_1 , as illustrated in the correctness analysis, $s_{j,k}$ is either equal to the receiver's choice $\hat{s}_{i,k}$ or a random function output. Because the honest receiver samples $\hat{s}_{i,k} \leftarrow \mathbb{F}_{2^{\ell_2}}$ uniformly at random, every $s_{j,k}$ in Hyb_1 is perfectly uniform and independent of $\mathcal{A}$'s view. Thus, the views in Hyb_1 and Hyb_2 are perfectly identical.
- Hyb_3 : This hybrid behaves identically to Hyb_2 , except that instead of passing honest receiver's queries to $\mathcal{F}_{\text{OPPRF}}$ (Step 3), Sim_S receives the programmed points $\{(k\|\tilde{q}_{j,k,t}, r_{j,k})\}_{j \in [m], k \in [d], t \in [u]}$ from $\mathcal{A}$, samples a random function F such that $F(k\|\tilde{q}_{j,k,t}) = r_{j,k}$ for $j \in [m], k \in [d], t \in [u]$, and sends O^F to $\mathcal{A}$. Since Hyb_2 also returns O^F according to these programmed points, the views in Hyb_2 and Hyb_3 are perfectly identical.

The final hybrid Hyb_3 is exactly the simulated execution of Sim_S . Therefore, the simulated view is perfectly indistinguishable from the real execution.

Corrupted Receiver. We construct a simulator Sim_R for a semi-honest corrupted receiver. Given the receiver's input W and the ideal output set Z from $\mathcal{F}_{\text{FPSI}}$, Sim_R runs $\mathcal{A}$ and proceeds as follows:

- (1) *Simulating $\mathcal{F}_{\text{OPPRF}}$ (Step 3)*: After receiving the $\mathcal{A}$'s queries $\{k\|\tilde{w}_{i,k,t}\}_{i \in [n], k \in [d], t \in [u']}$, Sim_R samples uniformly random values $\{\hat{r}_{i,k,t}\}_{i \in [n], k \in [d], t \in [u']}$, and returns them to $\mathcal{A}$.
- (2) *Simulating $\mathcal{F}_{\text{OPPRF}}$ (Step 4)*: After receiving the programmed points $\{(k\|\hat{r}_{i,k,t}, \hat{s}_{i,k})\}_{i \in [n], k \in [d], t \in [u']}$ from $\mathcal{A}$, Sim_R samples a random function F' such that $F'(k\|\hat{r}_{i,k,t}) = \hat{s}_{i,k}$ for $i \in [n], k \in [d], t \in [u]$. Sim_R returns $O^{F'}$ to $\mathcal{A}$.
- (3) *Simulating $\mathcal{F}_{\text{WLPSI}}$ (Step 6)*: After receiving $\mathcal{A}$'s input $\{\hat{s}_i\}_{i \in [n]}$, for every point $\mathbf{q}_j \in Z$, Sim_R identifies unique $i^* \in [n]$ such that $\text{dist}_\infty(\mathbf{w}_{i^*}, \mathbf{q}_j) \leq \delta$. Sim_R constructs the output set $I := \{(\hat{s}_{i^*}, \mathbf{q}_j) \mid \mathbf{q}_j \in Z\}$ and returns I to $\mathcal{A}$.

We demonstrate the security of the simulation through a sequence of hybrid games:

- Hyb_0 : This is the real protocol execution.
- Hyb_1 : This hybrid behaves identically to Hyb_0 except the simulation of $\mathcal{F}_{\text{WLPSI}}$ (Step 6). After receiving $\mathcal{A}$'s input $\{\hat{s}_i\}_{i \in [n]}$,

for each $\mathbf{q}_j \in Z$, Sim_R identifies the unique $i^* \in [n]$ such that $\text{dist}_\infty(\mathbf{w}_{i^*}, \mathbf{q}_j) \leq \delta$. This uniqueness is guaranteed by the globally disjoint assumption. Sim_R constructs the output set $I := \{(\hat{s}_{i^*}, \mathbf{q}_j) \mid \mathbf{q}_j \in Z\}$, and returns I to $\mathcal{A}$. The only difference between Hyb_0 and Hyb_1 occurs if the protocol produces an incorrect evaluation. As established in the correctness analysis, the probability is statistically negligible. Thus, the views in Hyb_0 and Hyb_1 are statistically indistinguishable.

- Hyb_2 : This hybrid behaves identically to Hyb_1 except that instead of invoking $\mathcal{F}_{\text{OPPRF}}$ (Step 4), after receiving the programmed points $\{(k\|\hat{r}_{i,k,t}, \hat{s}_{i,k})\}_{i \in [n], k \in [d], t \in [u']}$ from $\mathcal{A}$, Sim_R samples a random function F' such that $F'(k\|\hat{r}_{i,k,t}) = \hat{s}_{i,k}$ for $i \in [n], k \in [d], t \in [u]$ and returns $O^{F'}$ to $\mathcal{A}$. Since Hyb_1 also returns $O^{F'}$ according to these programmed points, the views in Hyb_1 and Hyb_2 are perfectly identical.
- Hyb_3 : This hybrid behaves identically to Hyb_2 , except that instead of passing the honest sender's points to $\mathcal{F}_{\text{OPPRF}}$ (Step 3), Sim_R receives $\mathcal{A}$'s queries $\{k\|\tilde{w}_{i,k,t}\}_{i \in [n], k \in [d], t \in [u']}$ and returns uniformly random values $\{\hat{r}_{i,k,t}\}_{i \in [n], k \in [d], t \in [u']} \leftarrow \mathbb{F}_{2^{\ell_1}}$. As illustrated in the correctness analysis, in Hyb_2 , $\mathcal{A}$ receives either the sender's payload $r_{j,k}$ or a random function evaluation. Because the honest sender samples $r_{j,k} \leftarrow \mathbb{F}_{2^{\ell_1}}$ uniformly at random, every returned evaluation is uniformly distributed. Thus, the views in Hyb_2 and Hyb_3 are perfectly identical.

The final hybrid Hyb_3 is exactly the simulated execution of Sim_R . Therefore, the simulated view is statistically indistinguishable from the real execution. $\square$

E.6 Proof of Theorem 7.3

PROOF. The correctness of the protocol is directly derived from the correctness of the L_∞ protocol for high-dimensional sets and the low-dimensional protocol, and hence we omit it here. Below, we mainly prove the security of the protocol by constructing simulators for both a corrupted sender and a corrupted receiver.

Corrupted Sender. We construct a simulator Sim_S for a semi-honest corrupted sender. Given the sender's input Q , Sim_S runs the adversary $\mathcal{A}$ and proceeds as follows:

- (1) *Simulating fuzzy mapping (Step 1)*: Sim_S invokes the simulator for the corrupted sender in the $\Pi_{\text{FPSI-high}}^{L_\infty}$ protocol and sends the resulting transcript to $\mathcal{A}$.
- (2) *Simulating $\mathcal{F}_{\text{OPPRF}}$ (Step 5)*: After receiving the programmed points $\{L_i\}_{i \in [n']}$ from $\mathcal{A}$, Sim_S samples a random function G programmed on $\{L_i\}_{i \in [n']}$. Sim_S sends O^G to $\mathcal{A}$.
- (3) *Simulating $\mathcal{F}_{\text{FMatch}}$ and $\mathcal{F}_{\text{OT}}$ (Steps 6-7)*: Sim_S receives its shares $\mathbf{g}_i$ to $\mathcal{F}_{\text{FMatch}}$ and $(\perp, \mathbf{g}_i)$ to $\mathcal{F}_{\text{OT}}$ from $\mathcal{A}$, and returns nothing.

We demonstrate the security of the simulation through a sequence of hybrid games:

- Hyb_0 : This is the real protocol execution.
- Hyb_1 : This hybrid behaves identically to Hyb_0 except the simulation of $\mathcal{F}_{\text{FMatch}}$ and $\mathcal{F}_{\text{OT}}$ (Steps 6-7). Sim_S receives $\mathcal{A}$'s inputs $\mathbf{g}_i$ to $\mathcal{F}_{\text{FMatch}}$ and $(\perp, \mathbf{g}_i)$ to $\mathcal{F}_{\text{OT}}$ and returns nothing,

which perfectly simulates the functionalities' behavior. Thus, the views in Hyb_0 and Hyb_1 are perfectly identical.

- Hyb_2 : This hybrid behaves identically to Hyb_2 except the simulation of $\mathcal{F}_{\text{OPPRF}}$ (Step 5). After receiving the programmed points $\{L_i\}_{i \in [n']}$ from $\mathcal{A}$, Sim_S samples a random function F programmed on $\{L_i\}_{i \in [n']}$. Sim_S sends O^G to $\mathcal{A}$. Since Hyb_1 also returns O^G according to these programmed points, the views in Hyb_1 and Hyb_2 are perfectly identical.
- Hyb_3 : This hybrid behaves identically to Hyb_2 , except that Sim_R invokes the receiver simulator for the $\Pi_{\text{FPSI-high}}^{L_\infty}$ protocol and sends the transcript to $\mathcal{A}$. The perfect indistinguishability of the views in Hyb_2 and Hyb_3 follows from the security of the L_∞ protocol.

The final hybrid Hyb_3 is exactly the simulated execution of Sim_S . Therefore, the simulated view is perfectly indistinguishable from the real execution.

Corrupted Receiver. We construct a simulator Sim_R for a semi-honest corrupted receiver. Given the receiver's input W and the ideal output set Z from $\mathcal{F}_{\text{FPSI}}$, Sim_R runs $\mathcal{A}$ and proceeds as follows:

- (1) *Simulating fuzzy mapping (Step 1)*: Sim_R invokes the simulator for the corrupted receiver in the $\Pi_{\text{FPSI-high}}^{L_\infty}$ protocol, purposefully omitting the final intersection delivery. Sim_R sends the transcript to $\mathcal{A}$
- (2) *Simulating $\mathcal{F}_{\text{OPPRF}}$ (Step 5)*: After receiving queries from $\mathcal{A}$, Sim_R samples uniformly random values $\hat{g}_i \leftarrow \mathbb{Z}_{2^{\ell'}}^d$ for all $i \in [n']$ and returns them to $\mathcal{A}$.
- (3) *Simulating $\mathcal{F}_{\text{Match}}$ and $\mathcal{F}_{\text{OT}}$ (Steps 6-7)*: For each bin $i \in [n']$, Sim_R identifies if there are a mapped element $T_w[i] = (\text{id}_{w_i} \parallel \tau, w_i)$ and a $q \in Z$ such that $\text{dist}_p(q, w_i) \leq \delta$. If such a match exists, Sim_R sets the simulated match bit $b_i := 1$ and constructs the OT payload as $z_i := q - \hat{g}_i \bmod 2^{\ell'}$. Otherwise (or if the bin is empty), Sim_R sets $b_i := 0$ and $z_i := \perp$. After receiving $\mathcal{A}$'s inputs, Sim_R returns $\{b_i\}_{i \in [n']}$ and $\{z_i\}_{i \in [n']}$ to $\mathcal{A}$ as the outputs of the functionalities.

We demonstrate the security of the simulation through a sequence of hybrid games:

- Hyb_0 : This is the real protocol execution.
- Hyb_1 : This hybrid behaves identically to Hyb_0 except the simulation of $\mathcal{F}_{\text{Match}}$ and $\mathcal{F}_{\text{OT}}$ (Steps 6-7). For each bin $i \in [n']$, if $T_w[i] = (\text{id}_{w_i} \parallel \tau, w_i)$, Sim_R checks if there exists an ideal match $q \in Z$ such that $\text{dist}_p(q, w_i) \leq \delta$. By the globally disjoint assumption, w_i can match at most one $q \in Z$, meaning this check is unambiguous. If a match exists, Sim_R sets the simulated match bit $b_i := 1$ and constructs the OT payload as $z_i := q - \hat{g}_i \bmod 2^{\ell'}$. Otherwise, Sim_R sets $b_i := 0$ and $z_i := \perp$. Sim_R returns $\{b_i\}_{i \in [n']}$ and $\{z_i\}_{i \in [n']}$ to $\mathcal{A}$. In Hyb_0 , if $b_i = 1$, $\mathcal{A}$ obtains the sender's mask g_i from $\mathcal{F}_{\text{OT}}$. In the simulation, setting $z_i := q - \hat{g}_i$ implicitly defines $z_i = g_i$, perfectly mimicking the relationship between the $\mathcal{F}_{\text{OPPRF}}$ uniform masks and the $\mathcal{F}_{\text{OT}}$ payloads while embedding the true ideal intersection Z . The only difference between Hyb_0 and Hyb_1 occurs if the protocol produces an incorrect evaluation that is statistically negligible. Thus, the views in Hyb_0 and Hyb_1 are statistically indistinguishable.

- Hyb_2 : This hybrid behaves identically to Hyb_1 , except that instead of passing the honest sender's programmed points to $\mathcal{F}_{\text{OPPRF}}$ (Step 5), Sim_R returns uniformly random values $\hat{g}_i \leftarrow \mathbb{Z}_{2^{\ell'}}^d$ for all $i \in [n']$. In Hyb_1 , $\mathcal{A}$ receives $\hat{g}_i = q_j - g_i \bmod 2^{\ell'}$ if the identifiers match, or a random function output otherwise. Because the honest sender samples $g_i \leftarrow \mathbb{Z}_{2^{\ell'}}^d$ uniformly at random, the masked value $q_j - g_i$ is perfectly uniform and independent of q_j . Thus, all outputs received by $\mathcal{A}$ in Hyb_1 are uniformly distributed. Thus, the views in Hyb_1 and Hyb_2 are perfectly identical.
- Hyb_3 : This hybrid behaves identically to Hyb_2 , except that Sim_R invokes the receiver simulator for the $\Pi_{\text{FPSI-high}}^{L_\infty}$ protocol, purposefully omitting the final intersection delivery. Sim_R sends the transcript to $\mathcal{A}$. The perfect indistinguishability of the views in Hyb_2 and Hyb_3 follows from the security of the L_∞ protocol.

The final hybrid Hyb_3 is exactly the simulated execution of Sim_R . Therefore, the simulated view is statistically indistinguishable from the real execution. $\square$